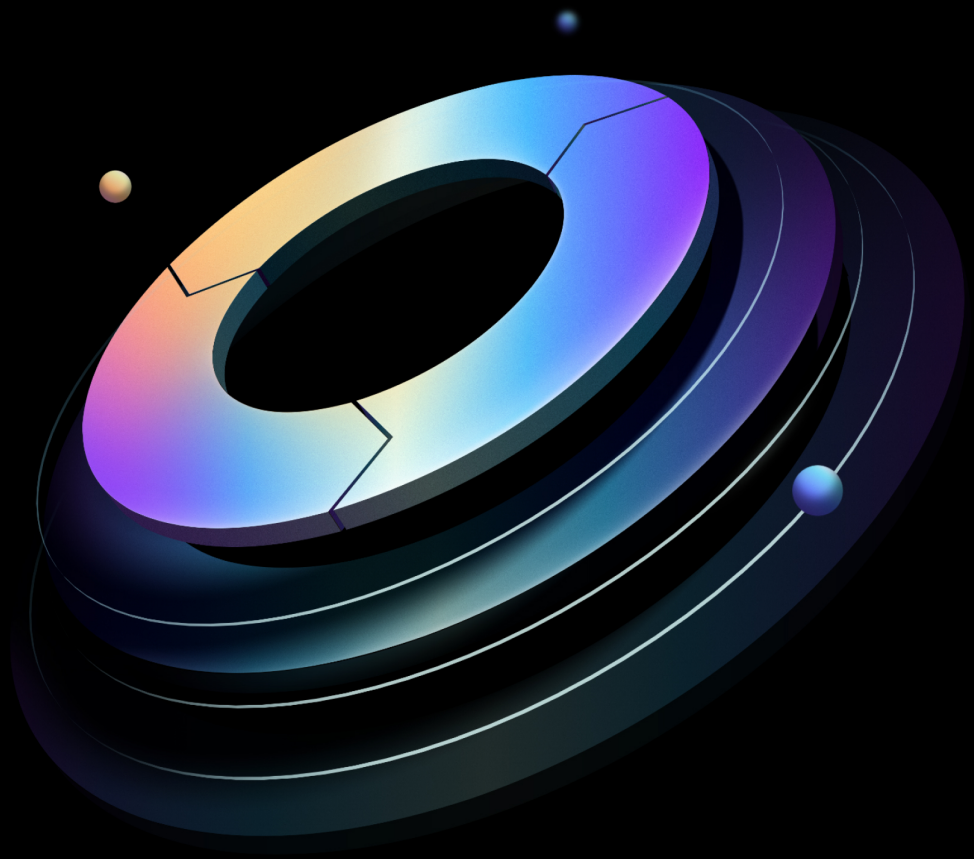
HashiCorp

Consul User Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **4**
1.1 Why use HashiCorp Validated Designs? . . . 4
1.2 Use cases covered . . . 5

2 Multi-platform service discovery **6**
2.1 Service registration overview . . . 6
2.2 Health checking . . . 7
2.2.1 Health check status . . . 9
2.3 User interface . . . 10
2.3.1 Services . . . 10
2.3.2 Service overview . . . 10
2.3.3 Instance detail . . . 11
2.3.4 Health check types . . . 12
2.4 Service registration workflows . . . 14
2.4.1 Configuration management (CM) service registration . . . 14
2.4.2 Application deployment service registration . . . 14
2.4.3 Terraform resource registration . . . 15
2.4.4 Consul catalog sync for Kubernetes . . . 15
2.4.5 Library or API registration . . . 16
2.5 Operational considerations . . . 17
2.5.1 ACLs . . . 17
2.5.2 Service naming conventions . . . 17
2.5.3 Leverage service meta and node meta fields . . . 17

3 Service discovery catalog **18**
3.1 DNS discovery . . . 18
3.2 Prepared queries . . . 19

4 Service mesh and gateways **20**
4.1 Service discovery and health monitoring . . . 20
4.1.1 Register client node and health check . . . 20
4.1.2 Register service and health checks . . . 22
4.1.3 Register external service with service discovery . . . 23

4.2 Service mesh and gateways . . . 26
4.2.1 Secure service communication with Consul service mesh and Envoy . . . 26
4.2.2 Connect services outside the service mesh . . . 27
4.2.3 API gateway - control access into service mesh . . . 30

5 Scale services on multiple infrastructures 35
5.1 Architectural elements . . . 38
5.1.1 Mesh gateways . . . 38
5.1.2 Cluster peering . . . 38
5.1.3 Consul dataplane - Kubernetes . . . 39
5.1.4 Transparent proxy . . . 42
5.1.5 Consul CNI . . . 42
5.2 Networking . . . 42
5.2.1 Load balancers for gateways . . . 43
5.2.2 Load balancers for VMs and Kubernetes . . . 43
5.3 Consul DNS . . . 44
5.3.1 Consul DNS on VM-based Consul clients . . . 44
5.3.2 DNS recursion . . . 45
5.3.3 Consul DNS on Kubernetes . . . 45
5.4 Service catalog sync . . . 45
5.5 High-level design . . . 46
5.6 References . . . 47

6 Multi-platform application deployment 47
6.1 Application deployment . . . 47
6.1.1 Prerequisites . . . 47
6.1.2 Deploy the Counting application on AKS . . . 49
6.1.3 Deploy the Dashboard application on EKS . . . 51
6.1.4 Dashboard service registration in AWS EC2 . . . 52
6.2 Nomad . . . 52

7 Deployment automation 53
7.1 Prerequisites . . . 53
7.2 Beginning CI/CD with Consul . . . 54
7.2.1 Managing software systems . . . 54
7.2.2 Considerations . . . 54

7.3 Software management of Consul . . . 55
7.3.1 Consul servers on VMs . . . 55
7.3.2 Services along with Consul client agents on VMs . . . 56
7.3.3 Consul on Kubernetes . . . 56

8 Services with Consul client agents 56
8.1 Immutable infrastructure approach . . . 57
8.2 Image management . . . 57
8.3 CI/CD implementation . . . 58
8.4 CI/CD workflow . . . 60
8.4.1 Checkout a “dev” branch . . . 60
8.4.2 Pull request process . . . 61
8.4.3 CI pipeline kickoff . . . 61
8.4.4 Check test results . . . 75
8.4.5 Pull request procedure . . . 75
8.4.6 Kick off the CI pipeline . . . 75
8.5 References . . . 77

9 Changelog 77
9.1 2026-08 . . . 77
9.2 2026-01 . . . 77
9.3 2025-06 . . . 77
9.4 2024-11 . . . 77

10 Credits 78

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

This User Guide is for platform and application teams consuming Consul Enterprise capabilities after the platform has been installed and handed over for operation. It covers service discovery, the service catalog, service mesh and gateways, multi-infrastructure service connectivity, deployment automation, and services with Consul client agents.

Note

Operator topics such as ACL governance, namespaces, admin partitions, cluster peering setup, observability, backup, restore, upgrades, and support readiness are covered in the Consul Administration Guide. Deployment architecture and install-time setup are covered in the Consul Installation Guide.

> **Warning**
>
> Please note that this document is a work in progress and is subject to change.

HashiCorp Validated Designs (HVDs) provide prescriptive guidance curated from our experience supporting numerous customer journeys with Consul Enterprise.

1.2 Use cases covered

This guide covers the Consul Enterprise capabilities that platform and application teams consume once the platform is installed and operational:

Use case	Summary
Multi-platform service discovery	Register services, configure health checks, and use the Consul service catalog across platforms.
Service discovery catalog	Use DNS discovery and prepared queries to discover registered Consul services.
Service mesh and gateways	Use Consul service mesh, gateways, and API gateways with HashiCups examples.
Multi-platform application deployment	Deploy example applications across Kubernetes, EC2, and Nomad using Consul.
Scale services on multiple infrastructures	Use Consul service mesh, mesh gateways, DNS, and catalog sync across multiple infrastructures.
Services with Consul client agents	Deploy services with Consul client agents on VMs using immutable infrastructure and CI/CD.
Deployment automation	Automate Consul-related delivery workflows with CI/CD, GitOps, and infrastructure tooling.

2 Multi-platform service discovery

The Consul service catalog is a core component of Consul's service discovery capabilities. Registering services in the catalog makes them discoverable by other services and tools within your infrastructure using regular DNS queries, even if they are unaware of Consul.

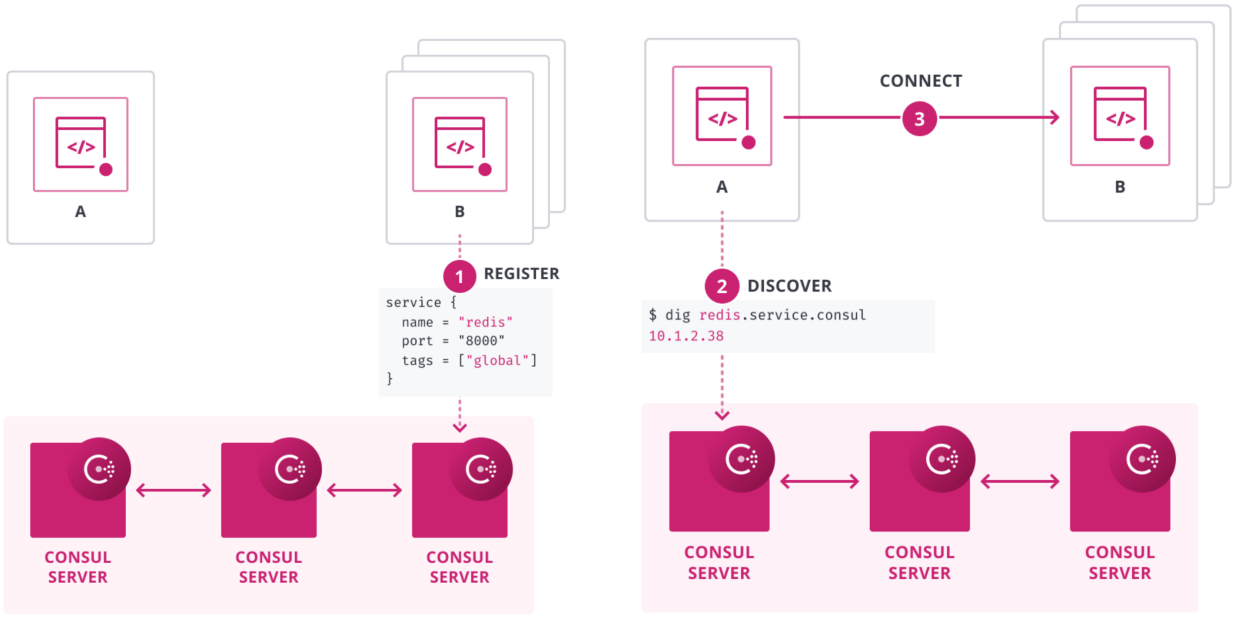


Figure 1: Service Discovery

2.1 Service registration overview

Below is an example of a service registration definition in HCL format that an agent can load at startup.

```
service {
    name = "hashicups"
    id = "hashicups"
    meta = {
        environment = "prod"
        version = "1.1"
        az = “us-east-1”a
    }
    port = 8000
}
```

We specify a name for the service, the port it is listening to, and any identifiable information about this specific service instance in the meta section so that it can be filtered against when querying the service. These filters help steer traffic to particular instances of a common service during application deployments or when feature flagging specific instances.

2.2 Health checking

Consul's health checking system is another important aspect of its service discovery capabilities. By understanding and effectively implementing health checks, you can ensure that only healthy instances of your services are discoverable and receive traffic. Here is a deeper dive into the health-checking workflow:

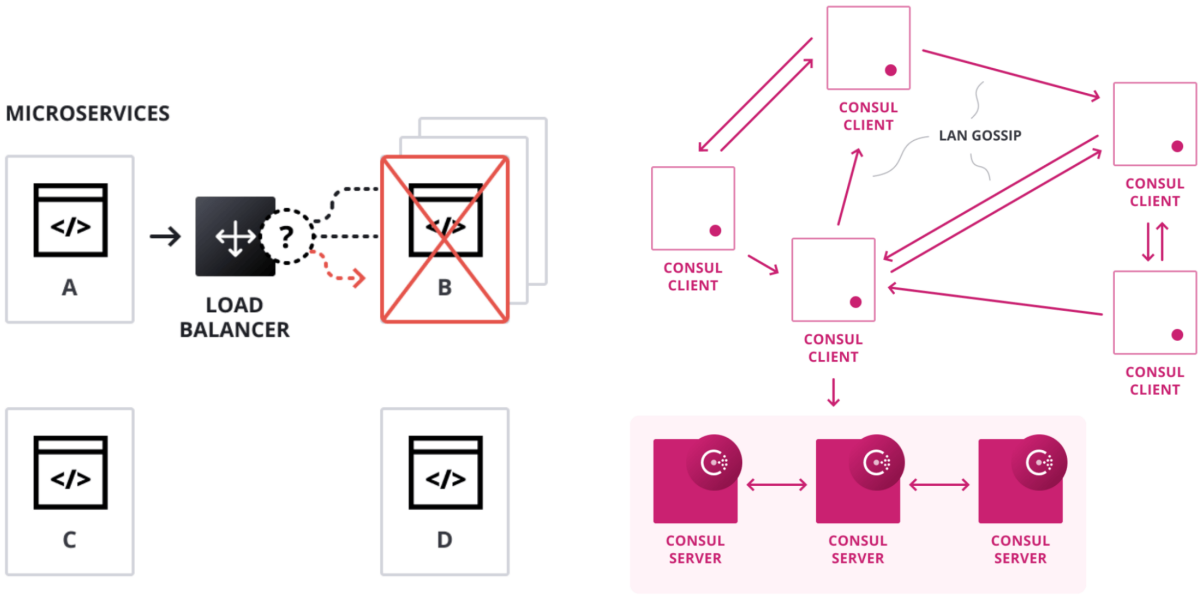


Figure 2: Service Discovery Health Checking

At its core, a health check in Consul is a method to determine the status of a service or a node. Consul supports multiple types of health checks, each tailored to different scenarios. When a service instance fails a health check, Consul automatically removes it from the list of healthy instances for that service, ensuring that consumers of that service don't send traffic to an unhealthy instance.

HashiCorp Consul's proficiency in health checking stems from its foundational architecture and

the protocols it employs. Central to Consul's efficiency is the Serf library, which utilizes the Gossip protocol to manage membership and detect node failures. Unlike many centralized systems, the Gossip protocol's decentralized nature allows for swift failure detection and minimizes single points of failure. This protocol also ensures efficient message broadcasting to all cluster nodes, ensuring rapid propagation of health state changes without overburdening the network. Moreover, its self-healing capabilities mean that if a node becomes unreachable, the protocol can route around it, ensuring uninterrupted propagation of health state information.

Another strength of Consul lies in its scalable health state distribution. Rather than relying on a polling mechanism, Consul adopts an event-based model. When there's a change in a service's health state, this change is immediately propagated to all relevant nodes, reducing unnecessary network traffic and ensuring timely updates.

Consul's integrated service discovery and health-checking approach offers a significant advantage. By combining these two functions, Consul ensures immediate updates to service discovery data when there's a change in a service's health state.

Application developers register health checks alongside their service registration definitions. You should use the same method to template or distribute your service registration definitions to include your applications' relevant health check definitions.

Below is an example of a service registration definition in HCL format that includes an HTTP health check.

```
service {
    name = "hashicups"
    id = "hashicups"
    meta = {
        environment = "prod"
        version = "1.1"
        az = “us-east-1”a
    }
    port = 8000
}
check = {
    id = "api"
    name = "HTTP API on port 5000"
    http = "https://localhost:5000/health"
    tls_server_name =  ""
    tls_skip_verify = false
    method = "POST"
    header = {
        Content-Type = ["application/json"]
    }
    body = "{\"method\":\"health\"}"
    disable_redirects = true
    interval = "10s"
    timeout = "1s"
}
```

2.2.1 Health check status

Health checks within Consul can return three different states when evaluating the health of your application.

Health check	Description
Passing	The service is healthy and can serve traffic.
Warning	The service might be experiencing issues but can still serve traffic.
Critical	The service is unhealthy and shouldn't receive traffic.

When querying via DNS, Consul will include services with health reporting in the *Passing* and *Warn-*

ing states by default. You can change this default behavior to only return *Passing* services with the `only_passing` configuration option in the `dns_config` configuration block. All three of these states can also be queried via the health API within the service catalog.

2.3 User interface

The Consul UI is a useful reference for providing an overall view of all registered services. The UI also provides more detailed views that can help investigation into failure states for every individual instance.

The UI can be enabled using the ui_config block on an agent by agent basis, the documentation for which is available here.

2.3.1 Services

The service tab shows a list of all services registered to the cluster, by default emphasizing services that have unhealthy instances.

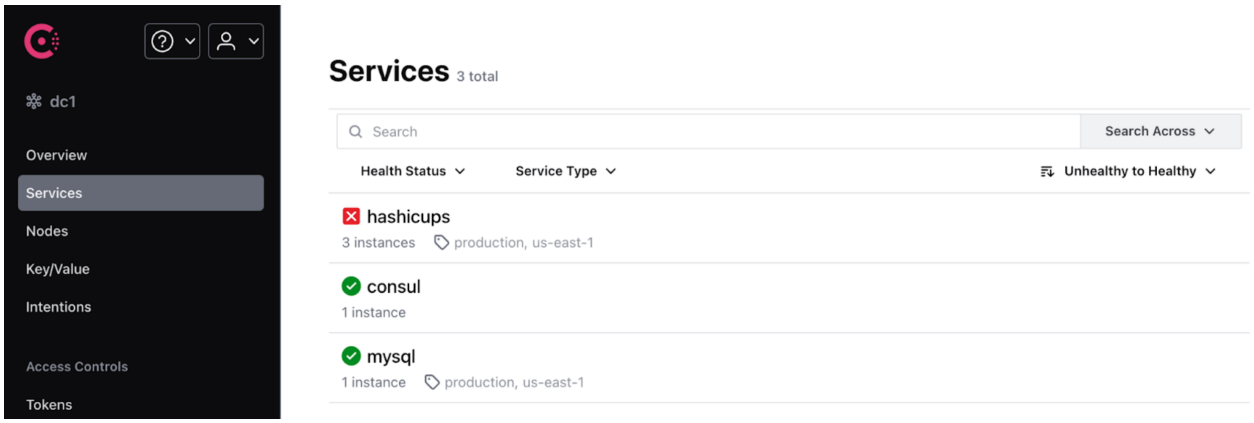


Figure 3: Services

2.3.2 Service overview

Selecting a specific service loads the next level of detail which provides a view of all instances of the chosen service and their individual health status.

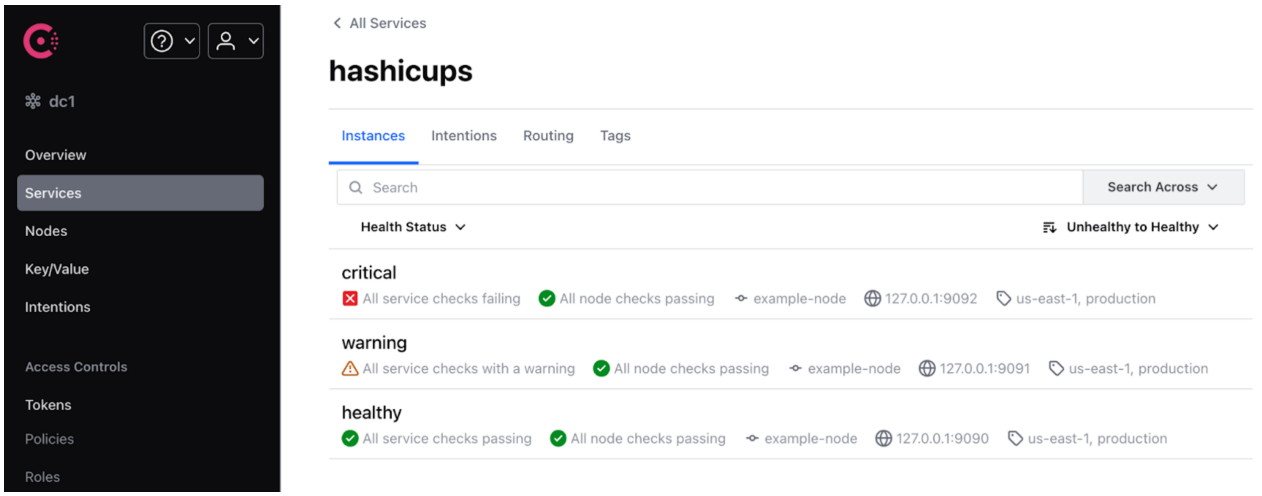


Figure 4: Service Detail

2.3.3 Instance detail

The response from each health check can be viewed in the instance detail view. In this example the instance in warning status can be seen to be responding with a 429 Too Many Requests HTTP code.

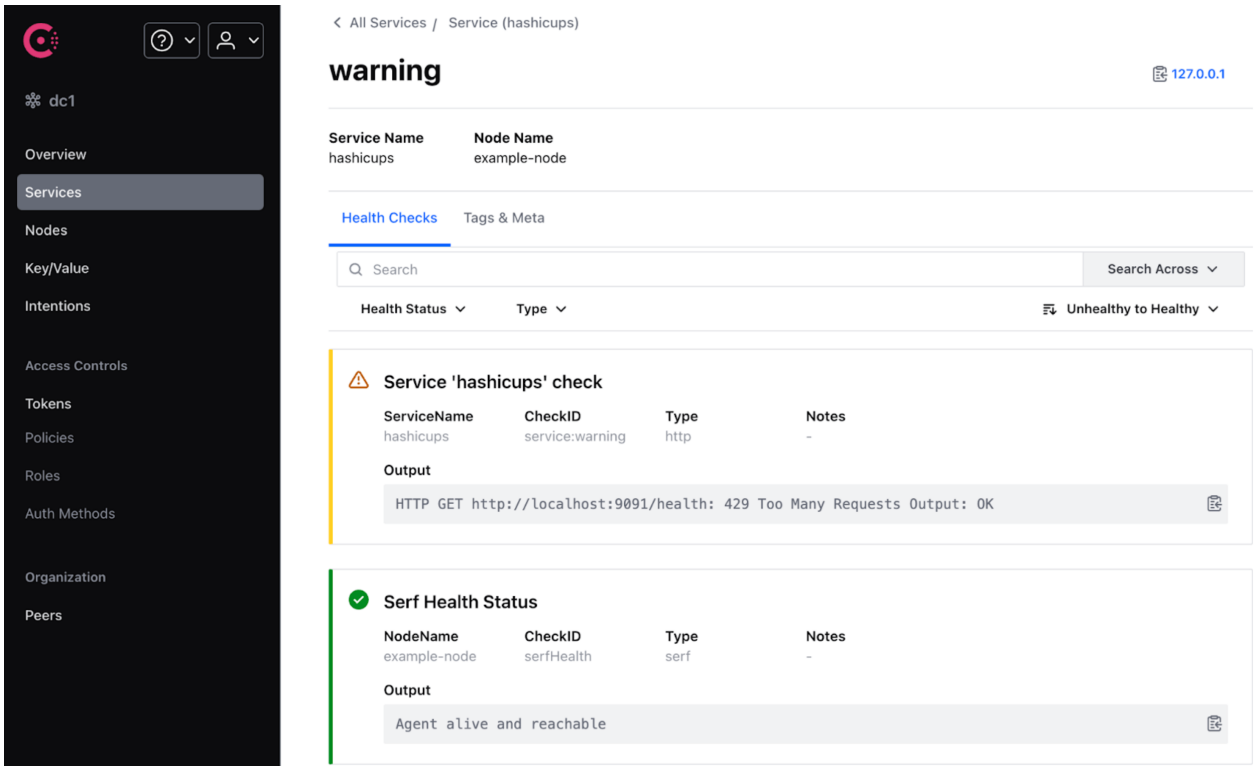


Figure 5: Instance Detail

2.3.4 Health check types

Health check types	Descriptions
HTTP	HTTP checks make an HTTP GET request to the specified URL and wait for the specified amount of time. HTTP checks are one of the most common types of checks.
TCP	TCP checks attempt to connect to an IP or hostname and port over TCP and wait for the specified amount of time.

Health check types	Descriptions
UDP	UDP checks send UDP datagrams to the specified IP or hostname and port and wait for the specified amount of time.
OSService	OSService checks if an OS service is running on the host. OSService checks support Windows services on Windows hosts.
Time-to-live (TTL)	Time-to-live (TTL) checks are passive checks that await updates from the service. If the check does not receive a status update before the specified duration, the health check enters a critical state.
Docker	Docker checks are dependent on external applications packaged with a Docker container that are triggered by calls to the Docker exec API endpoint.
gRPC	gRPC checks probe applications that support the standard gRPC health checking protocol.
H2ping	H2ping checks test an endpoint that uses http2. The check connects to the endpoint and sends a ping frame.
Alias	Alias checks represent the health state of another registered node or service.
Script	Script checks invoke an external application that performs the health check, exits with an appropriate exit code, and potentially generates output. Script checks carry a higher risk because they allow for code execution on client machines if someone has access to the client agent API.

Detailed configuration information for all health check types can be found here.

2.4 Service registration workflows

Application developers can register their services and health checks with the Consul Enterprise service catalog in several ways. We will highlight some of the most common methods to ensure that, as an operator, you understand the different approaches available to your consumers as they start adopting Consul Enterprise.

2.4.1 Configuration management (CM) service registration

Many organizations use configuration management tools like Chef, Ansible, or Puppet to manage their infrastructure. These tools can be leveraged to register services in Consul.

Workflow:

1. First, define the service registration and health check definition in JSON or HCL format
2. Use the templating functionality of your configuration management tool to render the service registration definition with any relevant variables substituted into the Consul configuration directory. Configuration is loaded in lexical order from the Consul configuration directory.
3. Trigger a Consul service reload either by using the `consul reload` CLI command, which is non-disruptive and keeps the agent running, or by restarting the Consul agent service.

2.4.2 Application deployment service registration

For application developers, it is often convenient to register services as part of the application deployment process.

Workflow:

1. First, define the service registration and health check definition in JSON or HCL format
2. Include the service registration definition with the application code or deployment artifacts in a place where the Consul agent can pick up the definition. Configuration is loaded in lexical order from the Consul configuration directory.
3. As part of the deployment automation (e.g., Jenkins, GitLab CI/CD, Spinnaker), use a script or tool to execute `consul reload` or restart the Consul agent service after deploying the application.

2.4.3 Terraform resource registration

Terraform, an Infrastructure as Code tool, can create resources like load balancers, databases, and VMs. These resources' IP addresses or DNS names can be registered in Consul as external services. This method does not support direct health checking of the service without using an external service monitor.

Workflow:

1. In your Terraform code, use the `consul_service` resource to define an external service alongside any infrastructure code that you want the service to reference (e.g., a load balancer or IP address from an instance)
2. Tie the attribute from the infrastructure resource that includes the IP address or DNS name to the `consul_node` resource associated with the external service.
3. When you apply your Terraform configuration, Terraform will create the infrastructure resource (e.g., a load balancer) and register the information from the corresponding `consul_service` resource in the Consul service catalog.

2.4.4 Consul catalog sync for Kubernetes

For organizations using Kubernetes, Consul provides a catalog sync feature, which automatically syncs services between Consul and Kubernetes.

Workflow:

1. Deploy one instance of the Consul catalog sync agent per Kubernetes cluster into a dedicated Partition inside your Consul Enterprise deployment using an external server configuration. Refer to our Installation Guide for Kubernetes specific deployment considerations.
2. Configure the admin Partition and Namespaces references, disable the connectInject functionality, and enable ACL management in your helm chart configuration. The connect inject functionality is not required for helm service discovery deployments that won't be leveraging sidecar containers for DNS proxying or mesh gateway components.
3. Configure the catalog sync agent to synchronize services from Kubernetes into Consul for specific Kubernetes namespaces.
4. Refer to the catalog sync helm reference for configuring settings for the service types that are applicable in your environment. eg: ClusterIP/NodePort/Ingress Controller

Example of configuration for step 2:

```
global:
    adminPartitions:
        enabled: true
        name: <non-default Partition name>
    enableConsulNamespaces: true
    acls:
        manageSystemACLs: true
connectInject:
    enabled: false
externalServers:
    enabled: true
    hosts: ["<Consul API destination>"]
    httpsPort: 8501
    grpcPort: 8503 # The grpc_tls port on Consul Servers
    tlsServerName: "<hostname tied to the https API>"
```

Example configuration for step 3:

```
syncCatalog:
    enabled: true
    toConsul: true
    toK8S: false
    k8sAllowNamespaces: ["specific-namespace1"”,specific-”namespace2]
    consulNamespaces:
        mirroringK8s: true
    addK8SNamespaceSuffix: false
    syncClusterIPServices: null
    ingress:
        enabled: null
        loadBalancerIPs: null
    nodePortSyncType: null
    aclSyncToken:
        secretName: null
        secretKey: null
```

2.4.5 Library or API registration

Several existing libraries for programming languages or application platforms include native support for registering and discovering services within the Consul service catalog. If there isn't library support directly in your application programming language, the REST API for registering services can also be leveraged to register your services dynamically at application runtime.

REST Workflow:

1. First, define the service registration and health check definition as a JSON object within your application code.
2. Use the agent catalog API to register the service to your local Consul agent where your application is running.

2.5 Operational considerations

As you begin working with application teams to register their services into Consul Enterprise, there are several critical operational considerations to think about to prepare each consumer from the beginning.

2.5.1 ACLs

The primary ACL type required for service registration is the `service:write` permission tied to the specific service name you are registering. If creating policies or roles for a large group of consumers who share a particular Namespace, you can summarize services using prefixes and then grant `service_prefix:write` with the appropriate prefix match for a group of services.

2.5.2 Service naming conventions

Maintain a consistent naming convention for your services across platforms and regions. This will help simplify the consumer workflow and create failover patterns with more advanced Consul features that can provide failover for similar services deployed in multiple places.

When breaking out Consul datacenters for different teams, you should leverage the enterprise namespace capabilities outlined in the initial configuration section or include prefixes in your service names based on application team names to ensure they are unique.

2.5.3 Leverage service meta and node meta fields

It would be best if you leveraged the meta field when registering services to include rich key-value pair information and associate it directly with your service instance. Meta values are preferred over tags because it allows easier filtering against the distinct keys or values.

In addition to adding meta values directly to your service instances, you can also apply meta values to the consul agents where the services are running in the `node_meta` configuration block. These meta values allow you to create filters based on specific characteristics of the underlying infrastructure where the service is running (e.g. Operating System, Kernel Version), or unique attributes of the individual service instance (e.g. Version, Feature Flags).

One consideration is that tags are directly exposed when using the DNS query API by including them as a prefix when querying service names. To do more advanced filtering against meta and node meta values, leverage the prepared query functionality outlined in the service catalog discovery section.

3 Service discovery catalog

Once services have been registered into the catalog with their respective health checks, application consumers can dynamically discover results to dial these services directly. As services scale up or down, the service catalog becomes the source of truth for finding healthy service instances using DNS.

3.1 DNS discovery

Once you have configured DNS forwarding following the recommendations in the *initial configuration* section, you can discover services and dial them directly using Consul's domain and query syntax, as shown below.

```
[<tag>.]<service>.service[.<namespace>.ns][.<partition>.ap].<domain>
```

- The TLD part <domain> is configured as `.consul` by default.
- The *tag*, *namespace*, and *partition* segments are optional.
- By default, all service lookups use the default namespace within the partition of the Consul agent that received the DNS query.

Consul server agents reside in the default partition. If your DNS queries target Consul server agents via DNS forwarding, you must explicitly specify the partition of the target service when querying for services in partitions other than "default".

For more information on performing DNS queries against the service catalog, refer to the documentation.

3.2 Prepared queries

To perform more advanced dynamic queries, application developers should leverage prepared queries. Prepared queries enable you to register a service query that provides more complex filtering than you could achieve using a DNS query.

Some restrictions apply to prepared queries:

- they can be defined only via Terraform or the API,
- they can be defined only in the default partition of a Consul datacenter.

You can not create prepared queries outside of the default partition. But, you can target services or service sameness groups located within other partitions.

When creating prepared queries that target services located in different local partitions within the same Consul datacenter, you do not need to export these services to the default partition as you would for services that exist in remote peers as long as your consumer Consul token has `service:read` permission across the partitions.

Prepared queries provide a rich set of lookup features. You can create queries that filter by including or excluding multiple tags, filter by service meta values, or filter services running on nodes based on node metadata defined inside the Consul client configuration.

Another important consideration is that you can define a TTL within your prepared query definition to reduce the load on Consul servers and anything in the DNS forwarding path. Prepared queries also offer rich failover capabilities, covered in the Multi-cluster and multi-tenant deployments section of the Consul Administration Guide, which covers running Consul Enterprise across multiple regions.

This is an example of a prepared query:

```
{
    "Name": "my-query",
    "Service": {
        "Service": "redis",
        "Near": "node1",
        "OnlyPassing": true,
        "Tags": ["primary", "!experimental"],
        "NodeMeta": { "instance_type": "m3.large" },
        "ServiceMeta": { "environment": "production" }
    },
    "DNS": {
        "TTL": "10s"
    }
}
```

Below is an example DNS query to discover services leveraging the above-prepared query.

```
my-query.query.<domain>
```

For detailed information on creating and reading the configuration of existing prepared queries, see here.

4 Service mesh and gateways

This guide uses the HashiCups as an example service applications registered on a Kubernetes deployment or pod and node with client agent to demonstrate the following use cases.

4.1 Service discovery and health monitoring

4.1.1 Register client node and health check

You can register nodes by deploying client agent configuration in the config directory with TF modules or with CLI and and start the client agents.

```
data_dir = "/etc/consul.d/consul-`hostname`"
node_name = "consul-`hostname`"
log_level = "${consul_log_level}"
datacenter = "${consul_datacenter}"
primary_datacenter = "${consul_primary_datacenter}"
encrypt = "${gossip_encryption_key}"
server = false
license_path = "${consul_license_path}"
enable_debug = true
ui_config {
  enabled = true
}
partition = "partition-1"
ports {
      serf_lan = 8302
    }
client_addr = "0.0.0.0"
retry_join = ["provider=aws region=us-west-2 tag_key=consul tag_value=consul"]
connect {
  enabled = true
}
rpc {
  enable_streaming = false
}
advertise_addr = "$INTERNAL_IP_ADDRESS"

# TLS config
tls {
 defaults {
verify_incoming = true
verify_outgoing = true
ca_file = "/etc/consul.d/tls/consul-ca.pem"
ca_path = "/etc/consul.d/tls"
cert_file = "/etc/consul.d/tls/consul-cert.pem"
key_file = "/etc/consul.d/tls/consul-key.pem"
}

# overrides tls.defaults path, if specified.
 https {
verify_incoming = true
verify_outgoing = true
}
 grpc {
verify_incoming = true
}

#Telemetry
telemetry {
  dogstatsd_addr = "127.0.0.1:8125"
  dogstatsd_tags = ["host-agent:consul-client"]
  prefix_filter = ["+consul.serf.member.flap", "+consul.serf.member.failed"]
}
```

```
consul agent -config-dir=${CONSUL_CONFIG_DIR} > /tmp/consul-client.log 2>&1 &
```

Consul also allows you to define node-level health-checks that are not tied to a specific service. These checks monitor the overall health of the node itself. Node-level health checks can be registered separately:

```
{
  "node": "consul-`hostname`",
  "check": {
    "script": "/usr/local/bin/check-node-health.sh",
    "interval": "30s"
  }
}
```

Here, a script-based health check runs every 30 seconds to check the health of the "consul-`hostname`".

4.1.2 Register service and health checks

To register a service in Consul, you typically use the `consul services register` command or by making an HTTP PUT request to the Consul agent's HTTP API. Service definition structure allows service health checks registered with service. Below are examples of both methods:

- Using `consul services register` command, we register counting service and its tcp health-check:

Product_db.json [VM]

```
{
  "service": {
    "ID": "product_db",
    "name": "product_db",
    "tags": ["backend", "product_db"],
    "port": 5432,
    "check": {
      "id": "product_db_check",
      "name": "Check DB health 5432",
      "tcp": "localhost:5432",
      "interval": "10s",
      "timeout": "1s"
    }
  }
}
```

```
consul services register prodct_db.json
```

- We can also register a service by making an HTTP PUT request to the Consul agent's API endpoint. Once the definition is saved you can register the dashboard service and HTTP health-check by running:

```
curl --request PUT --data @product_db.json http://127.0.0.1:8500/v1/agent/service/
    register
```

- Using Iaac approach using client agents, service definition files can be placed on disk that Consul can reload during agent start or with `consul reload` command.

4.1.3 Register external service with service discovery

Consul enables the registration and discovery of services internal to your infrastructure as well as external services: third-party SaaS services, and services running in other environments where it is not possible to run the Consul agent directly.

External services run on nodes where you cannot run a local Consul agent. These nodes might be inside your infrastructure (e.g. a mainframe, virtual appliance, or unsupported platform) or outside of it (e.g. a SaaS platform).

Because external services by definition don't have a local Consul agent, you can't register them with that agent or use it for health checking. Instead, you must register them directly with the catalog using the `/catalog/register` endpoint. We can use the /catalog/register endpoint for

registering external services or nodes but with `skip_node_update` to the API will not modify the node registration and assign specific service and checks to the existing node.

Typically external services are registered via nodes dedicated for that purpose so we'll continue our example using the Consul dev agent (localhost) as if it were running on a different node (e.g. "External Services") from the one where our internal web service is registered.

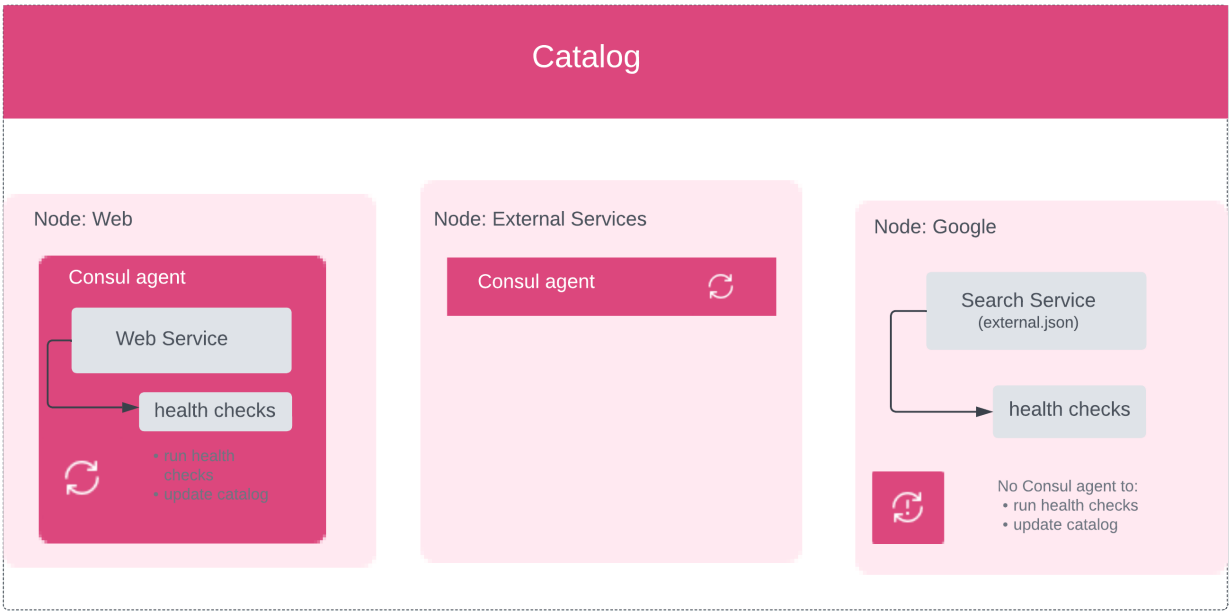


Figure 6: ESM

Use a PUT request to register the external service.

```
{
  "Node": "hashicorp",
  "Address": "learn.hashicorp.com",
  "NodeMeta": {
    "external-node": "true",
    "external-probe": "true"
  },
  "Service": {
    "ID": "learn1",
    "Service": "learn",
    "Port": 80
  },
  "Checks": [
    {
      "Name": "http-check",
      "status": "passing",
      "Definition": {
        "http": "https://learn.hashicorp.com/consul/",
        "interval": "30s"
      }
    }
  ]
}
```

```
curl --request PUT --data @external.json localhost:8500/v1/catalog/register
```

External service can be monitored with Consul ESM (External Services Monitor) that runs as a daemon alongside Consul in order to health check external nodes and update their status in the catalog.

Diagram shows how Consul ESM works with Consul to monitor the health of external services:

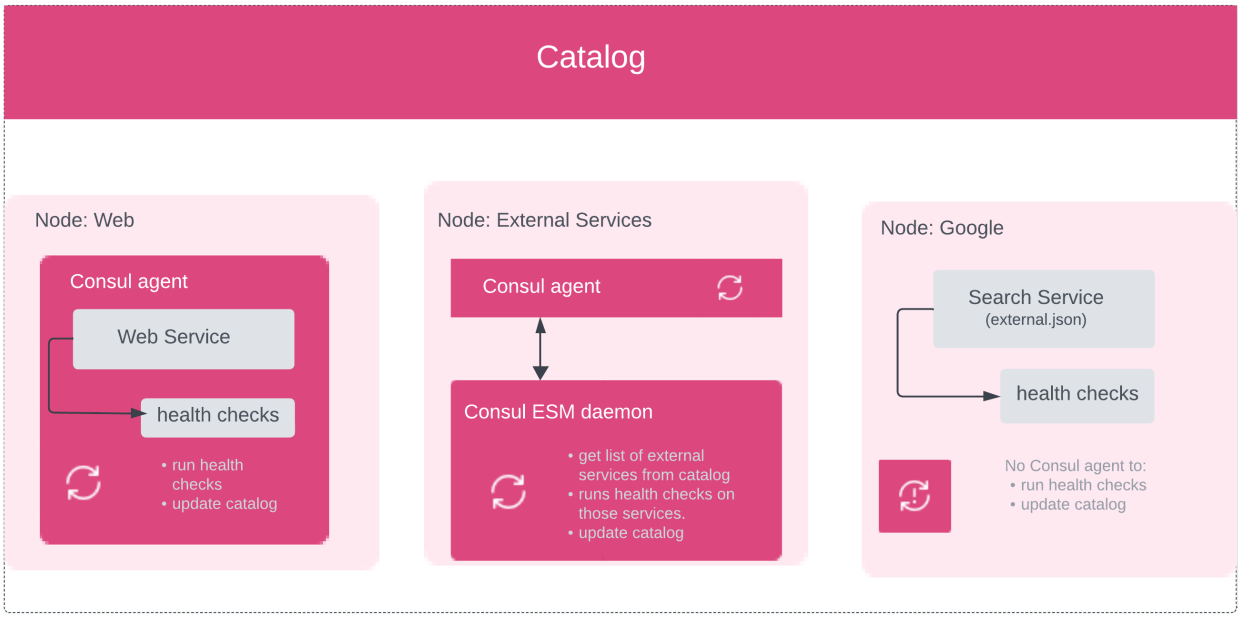


Figure 7: ESM2

4.2 Service mesh and gateways

4.2.1 Secure service communication with Consul service mesh and Envoy

Consul service mesh secures service-to-service communication with authorization and encryption. Applications can use sidecar proxies in a service mesh configuration to automatically establish TLS connections for inbound and outbound connections without being aware of the network configuration and topology. In addition to securing your services, Consul service mesh can also intercept data about service-to-service communications and surface it to monitoring tools.

We will register two services and their sidecar proxies in the Consul catalog and then start the services and sidecar proxies. Finally, we will demonstrate that the service-to-service communication is going through the proxies by stopping traffic with an "intention".

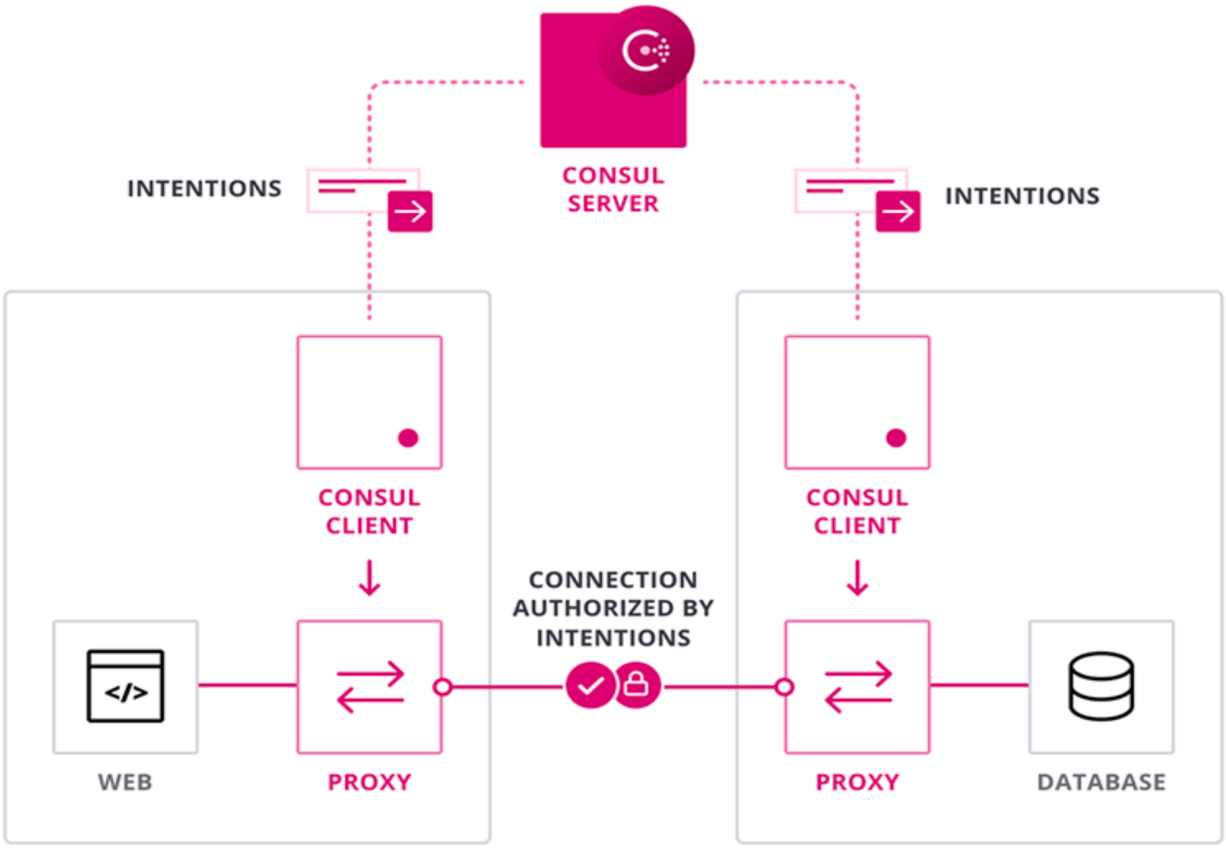


Figure 8: intentions

4.2.2 Connect services outside the service mesh

The simplest use case for terminating gateways is routing to external services in the same logical network as the Consul service mesh.

A managed service, such as Amazon RDS or a third-party service, that needs to be integrated into the service mesh. In this scenario, you will not be able to install Consul on the machine and you will only be able to access the service via the endpoint(s) it exposes.

In these situations, given the heterogeneous nature or location of these services, the gateways and the external services would be registered in the same Consul datacenter catalog. Traffic from services in the mesh would flow through their sidecar to the terminating gateway. The gateway will then forward the traffic, potentially unencrypted, to the destination service.

To terminate TLS connections, the terminating gateway will present leaf certificates for the services it represents. Internal mesh traffic to the gateway will be encrypted with mTLS and controlled by intentions as expected with Consul service mesh. Intentions reference the source and destination services and do not require knowledge of the gateway itself.

The routing between gateways and external services is determined by the `terminating-gateway` configuration entry. The configuration is associated with the name of a gateway service and will apply to all instances of gateways with that name across all federated Consul datacenters.

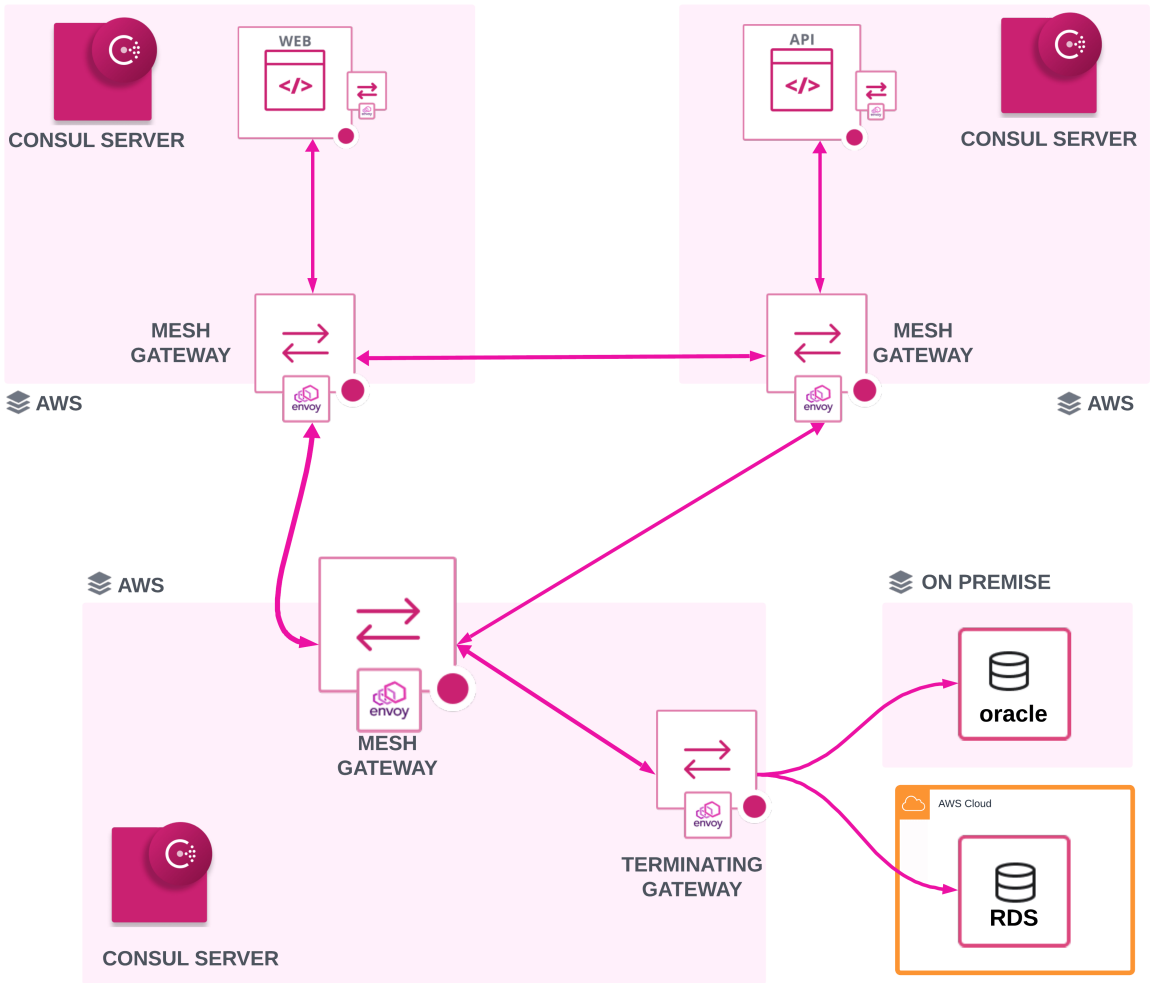


Figure 9: Terminating Gateways

Update helm chart with minimum required configuration to install terminating gateways.

```
global:
  name: consul
terminatingGateways:
  enabled: true
```

Registering the external services with Consul is a multi-step process:

- Register external services with Consul using `ServiceDefaults`, if `TransparentProxy` is enabled. Otherwise, you register the service as a node in the Consul catalog. Example below configure `ServiceDefaults` to register AWS RDS postgres instance with Consul Service Mesh.

```
apiVersion: consul.hashicorp.com/v1alpha1
  kind: ServiceDefaults
  metadata:
    name: example-rds
  spec:
    protocol: tcp
    destination:
      addresses:
        - "consul53.test.us-west-2.rds.amazonaws.com"
      port: 5431
```

- Update the terminating gateway ACL token if ACLs are enabled. Update the terminating gateway ACL role to have service:write permissions on all of the services being represented by the gateway.

Create a new policy that includes the write permission for the service you created.

```
service "example-rds" {
  policy = "write"
}
```

Apply terminating gateway `write-policy.hcl` and create ALC role.

```
$ consul acl policy create -name "example-rds-write-policy" -rules @write-policy.hcl

# Obtain the ID of the terminating gateway role.

$ consul acl role list -format=json | jq --raw-output '[.[] | select(.Name | endswith
    ("-terminating-gateway-acl-role"))] | if (. | length) == 1 then (. | first | .ID)
     else "Unable to determine the role ID because there are multiple roles matching
    this name.\n" | halt_error end'
<role_id>

# Update the terminating gateway ACL role with the new policy.

$ consul acl role update -id <role id> -policy-name example-rds-write-policy
```

- Create a `TerminatingGateway` resource to configure the terminating gateway after acl role update.

```
#terminating-gateway.yaml
apiVersion: consul.hashicorp.com/v1alpha1
kind: TerminatingGateway
metadata:
  name: terminating-gateway
spec:
  services:
    - name: example-rds
```

- Create a `ServiceIntentions` resource to allow access from services in the mesh to external service.

```
apiVersion: consul.hashicorp.com/v1alpha1
kind: ServiceIntentions
metadata:
  name: example-rds
spec:
  destination:
    name: example-rds
  sources:
    - name: api
      action: allow
```

4.2.3 API gateway - control access into service mesh

Consul API Gateway is a dedicated ingress solution for intelligently routing traffic to applications on your Consul service mesh. This provides a consistent method for handling inbound requests to

the service mesh from external clients.

Consul API Gateway takes all API calls from clients, then routes them to the appropriate service with request routing, composition, and protocol translation. Once Consul API Gateway becomes available for your services, you can use it for ingress, load balancing, modifying HTTP headers, and splitting traffic between multiple services based on weighted ratios.

Consul API Gateway can be enabled by defining the API gateway in the `ConnectInject` stanza as specified in consul server helm file.

Consul server helm

```
global:

  name: consul
  image: "hashicorp/consul-enterprise:1.16.X-ent"
  datacenter: default
  adminPartitions:
    enabled: true
    name: "default"
  acls:
    manageSystemACLs: true
  enableConsulNamespaces: true
  enterpriseLicense:
    secretName: consul-enterprise-license
    secretKey: license
    enableLicenseAutoload: true
  peering:
    enabled: true
  tls:
    enabled: true

server:
 replicas: 3
 bootstrapExpect: 3
 exposeService:
    enabled: true
    type: LoadBalancer
 extraConfig: |
    {
      "log_level": "TRACE"
    }

syncCatalog:
 enabled: true
 k8sAllowNamespaces: ["*"]
 consulNamespaces:
   mirroringK8S: true

connectInject:
  enabled: true
  transparentProxy:
    defaultEnabled: false
  consulNamespaces:
    mirroringK8S: true
  k8sAllowNamespaces: ['*']
  k8sDenyNamespaces: []
  apiGateway:
    managedGatewayClass:
      serviceType: LoadBalancer


meshGateway:
  enabled: true
  replicas: 3
  service:
    enabled: true
    type: LoadBalancer
```

Deploy API gateway

A complete API Gateway deployment consists of an API Gateway configuration and a routing configuration. It consists of multiple components that enable external traffic into your Consul service mesh. The configuration file specifies how Consul API Gateway will handle API calls from clients and how it will route them to the respective services with request routing, composition, and protocol translation.

`consul-api-gateway.yaml`

```
apiVersion: gateway.networking.k8s.io/v1beta1
kind: Gateway
metadata:
  name: api-gateway
  namespace: consul
spec:
  gatewayClassName: consul
  listeners:
    # options: HTTP or HTTPS
  - protocol: HTTP
     # options: 80 or 443 or custom
    port: 80
    name: http
    allowedRoutes:
      namespaces:
        # options: All or Same or Specific
        from: All
```

This configuration file also defines other objects needed for the deployment of the Consul API Gateway, such as `ReferenceGrant`, `ClusterRoleBinding` and `ClusterRole`. The reference grant lets API Gateway route traffic to services in different namespaces, and the RBAC ClusterRole objects let the API gateway interact with Consul datacenter resources.

```
#Apply consul-api-gateway.yaml to deploy the gateway
$ kubectl apply --filename api-gw/consul-api-gateway.yaml

# Verify the deployed gateway
$ kubectl get services --namespace=consul api-gateway

# Export API gateway external loadbalancer DNS
$ export APIGW_URL=$(kubectl get services --namespace=consul api-gateway -o jsonpath=
    '{.status.loadBalancer.ingress[0].hostname}') && echo $APIGW_URL
```

Create API gateway ingress route for the Frontend service

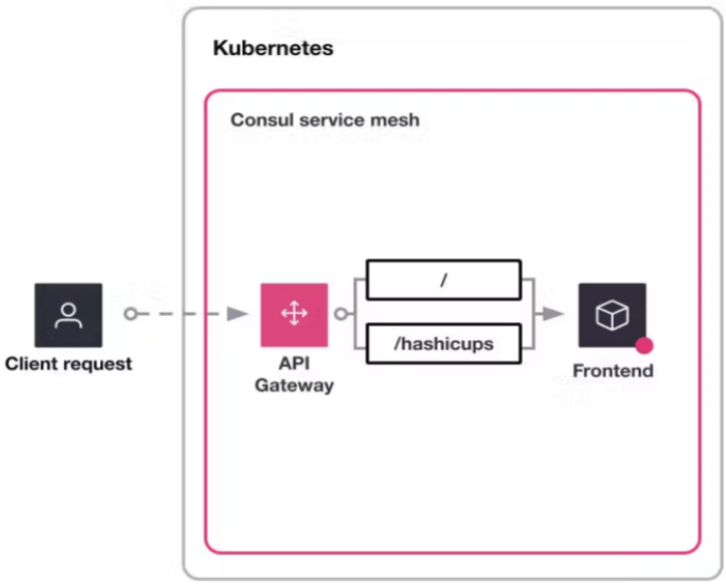


Figure 10: api gateway

```
apiVersion: gateway.networking.k8s.io/v1beta1
kind: HTTPRoute
metadata:
  name: route-hashicups
  namespace: default
spec:
  parentRefs:
  - name: api-gateway
    namespace: consul
  rules:
  - matches:
    - path:
        type: Exact
        value: /hashicups
    backendRefs:
    - kind: Service
      name: frontend
      namespace: default
      port: 3000
    filters:
      - type: URLRewrite
        urlRewrite:
          path:
            replacePrefixMatch: /
            type: ReplacePrefixMatch
```

5 Scale services on multiple infrastructures

> Note
>
> Namespaces, admin partitions, cluster peering, and mesh gateway governance are configured and managed by platform operators — see the Administration Guide: Multi-cluster and multi-tenant deployments.

Consul provides a way for organizations to secure service-to-service communications across numerous clouds and run-times based on service identity, instead of difficult-to-manage IP addresses. You are able to integrate with your preferred identity access management (IAM) system and OpenID Connect (OIDC) providers. This gives platform teams the ability to limit machine, database, application, and service account access based on policies and role assignments.

Implementing Consul's security and service communication features requires planning and incorporation of HashiCorp's Cloud Operating Model. It is especially important for developers to consider

the following constructs:

1. **Provision:** Leverage multi-cloud infrastructure provisioning with Terraform. Using Terraform is especially beneficial in large organizations that utilize on-premises and multi-cloud deployments. Taking an infrastructure-as-code (IaC) approach provides a reliable and consistent way to self-service infrastructure that is compliant and easily managed.
2. **Secure:** Adopt zero-trust practices by securing east-west communication with Consul across distributed network boundaries. Secure workload communications running on multiple runtimes with encryption, authorization, and Vault secret storage.
3. **Connect:** The networking layer transitions from being heavily dependent on the physical location and IP addresses of services to using a dynamic registry of services for discovery, segmentation, and composition.
4. **Execute:** Run applications on infrastructure which is provisioned on-demand using a scheduler. Use Consul to connect the applications securely.

While scaling, there will be scenarios where organizations will want to apply multi-tenancy across several clusters, all while sharing a Consul cluster between various clouds. In this document, you will walk through the following scenario:

- An organization has on-premises infrastructure, and has made heavy investments in VM-based virtualization. They have a single Consul cluster running on the virtual machines. Two partitions have been created: `frontend-team` and `backend-team`.
- The frontend team manages a dashboard application and relies on the backend team to provide data for the dashboard. The backend team chose to containerize their counting application by standing up Nomad infrastructure with Consul clients installed on each node, which register back with the Consul server cluster.
- Business has been increasing for the organization. They now have to support other geographical areas, and therefore need to scale to public cloud providers. The frontend team is choosing to deploy their dashboard in AWS, while the backend team has chosen to deploy their counting application in Azure.
- A third platform team is using Consul's cluster peering feature to join the two cloud-hosted clusters and existing on-premises environment, providing a failover path for the backend team's application.

> **Note**
>
> Admin partitions are only supported in Consul Enterprise.

The diagram shows how the business-supporting applications will be installed:

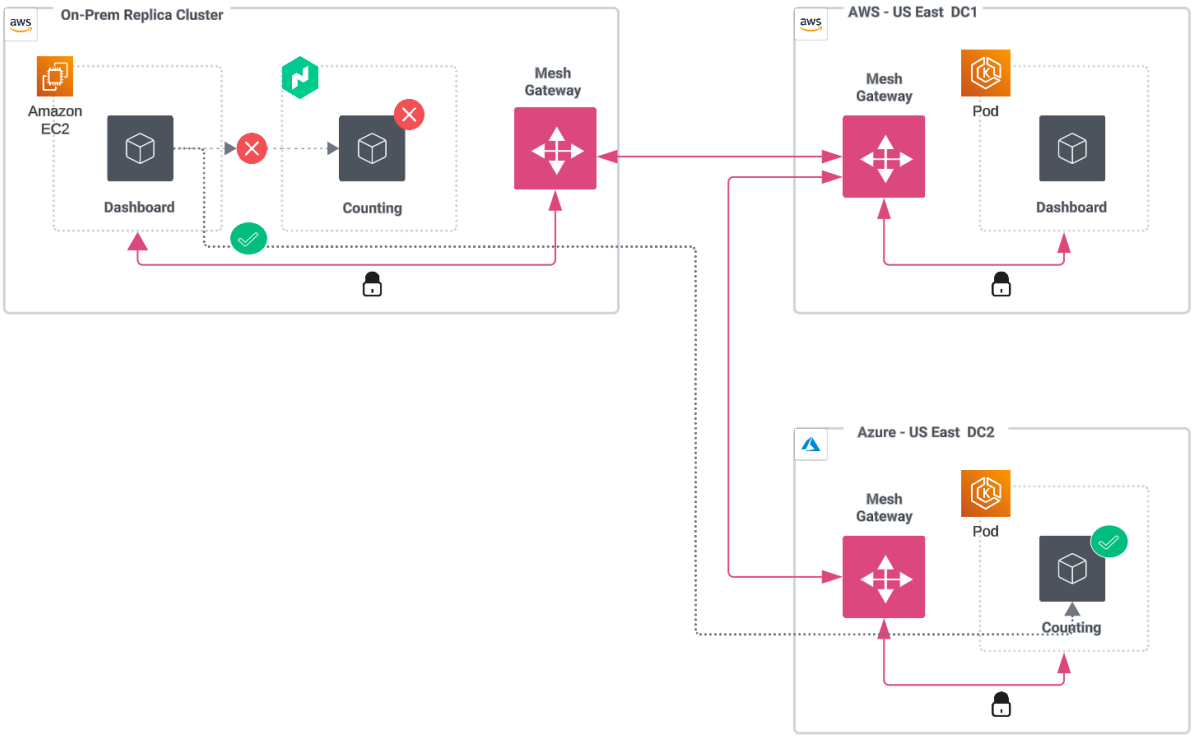


Figure 11: Business application installation

Notes on the diagram:

- On-premises dashboard application is install on VMs.
- On-premises counting application is installed on VMs with a Nomad runtime.
- Dashboard application is installed on EKS in AWS-US East DC1.
- Counting application is installed on AKS in an HA configuration in Azure-US East DC2.

5.1 Architectural elements

Before diving into the installation details, it is important to get introduced to the architectural elements that will enable you to achieve the targeted state architecture needed by your organization.

5.1.1 Mesh gateways

Consul clusters may run on different cloud providers or in multiple regions within the same provider's system. Consul's mesh gateways facilitate cross-cluster communication by encrypting outgoing traffic, decrypting incoming traffic, and routing traffic to healthy services. Mesh gateways are required to route service-to-service traffic between datacenters and handle control plane traffic from Consul servers. They are also used to route service-to-service traffic between admin partitions. These gateways sit at the edge of a cluster or on the public internet, and accept Layer 4 (L4) traffic with mutual TLS (mTLS) encryption.

Mesh gateways, when crossing network boundaries like the internet or regional boundaries with the same cloud service provider (CSP), should leverage the cloud provider's automatic load balancer configuration. For example, when services on Kubernetes are exposed via the type `LoadBalancer`, the cloud provider will automatically create a corresponding load balancer within the environment. Consul advertises the load balancer's address as the WAN interface for the mesh gateway.

In the context of this document, mesh gateways will be used for cluster peering and will not utilize Consul's traditional WAN federation model.

5.1.2 Cluster peering

Cluster peering uses various components of Consul's architecture to ensure secure service-to-service communication:

- A *peering token* contains a secret that facilitates secure communication when symmetrically shared between datacenters. By exchanging this token, server agents in a datacenter can authenticate requests from the corresponding peer, similar to how the gossip encryption key protects LAN and WAN gossip between Consul agents.
- A *mesh gateway* is responsible for routing encrypted traffic and requests to healthy services. For mesh gateways to function, Consul's service mesh capabilities must be activated. These gateways are specific to the admin partitions they are deploy in.

- An *exported service* makes a given upstream service discoverable by downstream services in the service catalogs of the consuming admin partitions or peers. These services must be explicitly defined within the `exported-services` configuration entry or through a Kubernetes custom resource.
- *Service intentions* authorize communication between services in the mesh. They provide identity-based access by authorizing connections based on the downstream service identity that is encoded in the TLS certificate used by the sidecar proxy when connecting to upstream services.

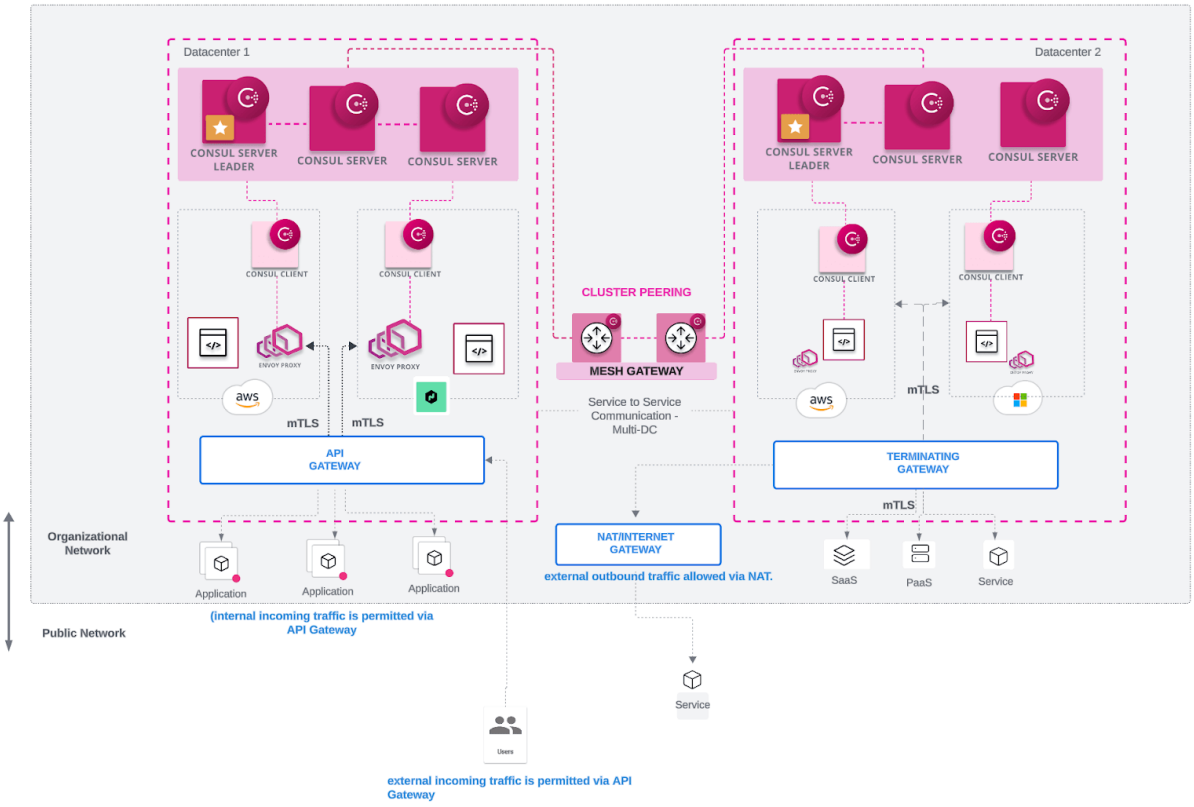


Figure 12: Cluster peering

5.1.3 Consul dataplane - Kubernetes

When deploying Consul service mesh, organizations will benefit from adding the Consul dataplane sidecar with built-in Envoy proxy capabilities. Consul dataplane was introduced in Consul v1.14

and replaces the Consul client agent with a lightweight agent. Consul dataplane does not use gossip, and relies on Kubernetes health checks to validate the health of a pod.

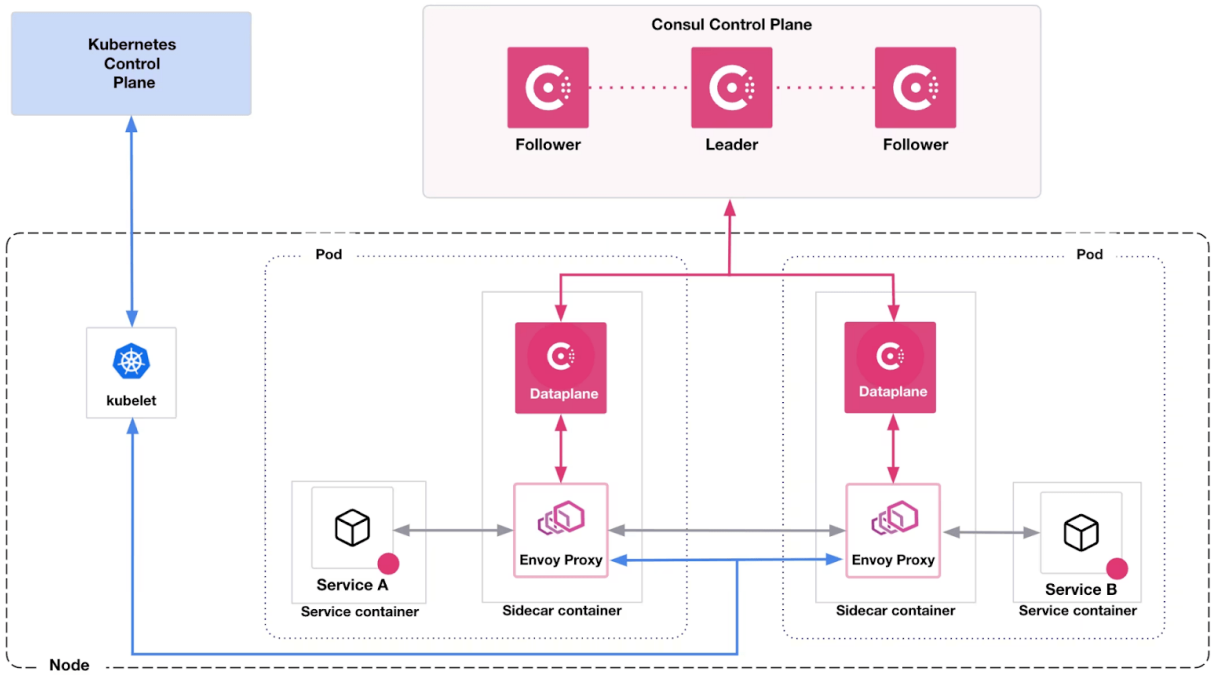


Figure 13: Consul dataplane

Consul dataplane has the following benefits:

- **Fewer network requirements:** Bidirectional network connectivity between nodes is not required for gossip communication. This has been replaced by a single gRPC connection to the Consul servers.
- **Simplified setup:** Because gossip communication is not require, there is no need for generating or rotating encryption keys, resulting in fewer tokens to track.
- Support for AWS Fargate and GKE Autopilot.

Organizations adopting Consul service mesh can find a compatibility matrix, list of supported features, and technical constraints in the Consul Installation Guide. The Helm values file given below shows the options that are recommended to turn on the Consul dataplane:

```yaml
global:
connectInject:
    replicas: 2
    consulNamespaces:
        mirroringK8S: true
    transparentProxy:
        defaultEnabled: true
    cni:
        enabled: true
        namespace: "consul"
        cniBinDir: "/var/lib/cni/bin"
        cniNetDir: "/etc/kubernetes/cni/net.d"
        multus: true
    metrics:
        defaultEnabled: true
        defaultEnableMerging: true
    resources:
        requests:
            memory: "200Mi"
            cpu: "100m"
        limits:
            memory: "500Mi"
            cpu: "500m"
    failurePolicy: "Ignore"
    k8sDenyNamespaces:
        - default
    sidecarProxy:
        resources:
        requests:
            memory: 100Mi
            cpu: 100m
        limits:
            memory: null
            cpu: null
    initContainer:
        resources:
            requests:
                memory: "25Mi"
                cpu: "50m"
            limits:
                memory: "150Mi"
                cpu: "50m"
    apiGateway:
        manageExternalCRDs: true
        managedGatewayClass:
            serviceType: LoadBalancer
```

5.1.4 Transparent proxy

The Consul service mesh transparent proxy simplifies service mesh adoption by allowing services to connect securely without modifying application code. It leverages Envoy as a sidecar proxy, automatically intercepting and managing service-to-service communication using mTLS, authorization policies, and traffic routing rules.

In a traditional Consul service mesh setup, services must be explicitly configured to communicate to upstream services via local ports that are bound by the Envoy proxy sidecar. With the transparent proxy, however, this requirement is removed. Consul automatically configures the network to intercept traffic from the application without requiring any changes to service configurations or code and forwards it to Envoy. This allows legacy applications or services that cannot be easily modified to participate in the service mesh with minimal effort.

5.1.5 Consul CNI

The Consul Container Network Interface (CNI) plugin is deployed to enable transparent proxy functionality for service mesh workloads, and to avoid the need for privileged permissions within application initialization containers.

```
connectInject:
  replicas: 2
  consulNamespaces:
    mirroringk8s: true
  transparentProxy:
    defaultEnabled: true
  cni:
    enabled: true
    namespace: "consul"
    cniBinDir: "/var/lib/cni/bin"
    cniNetDir: "/etc/kubernetes/cni/net.d"
    multus: true
```

5.2 Networking

Managing multiple cloud environments often introduces significant overhead for customers. Establishing connectivity between disparate cloud providers typically involves setting up VPN tunnels. These include examples such as IPsec VPNs that traverse the public internet, or dedicated circuits like AWS Direct Connect, Microsoft ExpressRoute, and GCP Cloud Interconnect.

Each of these methods presents its own set of complexities, including issues with scalability, fragmented visibility, and inconsistencies in security policies. Moreover, the requirement to avoid overlapping IP addresses further complicates configuration and management. Consul service mesh addresses these networking challenges by offering a cohesive networking model that unifies services across these varied and disconnected environments.

5.2.1 Load balancers for gateways

Consul provides load balancing capabilities by randomizing the DNS responses when queried by one service trying to reach another within the service mesh; this can be further extended to high availability, resilience and failover. Consul's DNS and load balancing capabilities help organizations reduce network costs and complexity by reducing the amount of load balancers and firewalls in a service communication path.

In a large-scale deployment - especially when connecting different network boundaries - there will be situations where there are services that have not yet been migrated to Consul, or fall under a different administrative control where Consul cannot be installed. These are two scenarios where load balancers will be required.

Organizations with regionally diverse datacenters or multiple CSPs will need load balancers for network translation between public and private subnets. A Network Load Balancer (NLB) operates at Layer 3 and Layer 4, offering IP/Port translation and managed traffic between cloud and on-premises environments.

5.2.2 Load balancers for VMs and Kubernetes

An on-premises or public cloud virtual machine- or container-based deployment will have load balancers deployed in the following scenarios:

- **Front-ending the service mesh at the north-south ingress point:** These load balancers are typically L4 and can be set up to load balance L4 traffic between multiple API gateways. These API gateways typically handle L7 traffic providing SSL termination, authentication, session persistence, and other balancing techniques like service splitting based on HTTP headers.
- **Exposing Consul's DNS endpoint for query by upstream DNS servers:** Consul exposes both TCP and UDP port 8600 for DNS queries against its service catalog. An L4 load balancer can be set up to proxy DNS queries on port 53 to port 8600 serviced by multiple Consul servers,

preferably to non-voting Consul servers. This will reduce the risk of overloading the voting servers.

- **Connecting mesh gateways between two regions or datacenters with different admin controls:** Mesh gateways operate at the edge of the mesh. They receive incoming or outgoing HTTP/TCP connections to services running in the mesh and for inter-cluster calls. To connect mesh gateways across different administrative boundaries, load balancers will be required to provide a static virtual IP, which can be used to route incoming traffic. You will see this, for example, in mesh gateways deployed across VPCs, where public-facing load balancers frontend multiple mesh gateways in a region.

The above scenarios are discussed further in subsequent sections, and will be covered in the installation modules provided.

5.3 Consul DNS

Consul offers a DNS interface for querying services registered in the Consul catalog, and is the main mode of service resolution for communication between Consul registered services. For a deployment at scale, companies will need to establish a DNS strategy for connecting services in the following states:

1. Services that are not yet discovered by Consul and are not part of the mesh (off-mesh), or
2. Services that are discovered by Consul and/or are part of the mesh (on-mesh).

This becomes important when workloads are running monolith or microservices-based applications on Kubernetes. It is also applicable when you have VM-based deployments leveraging on-premises infrastructure or public clouds. HashiCorp can provide guidance around how to manage DNS behavior for services running on-mesh, or how to configure Consul DNS.

5.3.1 Consul DNS on VM-based Consul clients

Consul DNS can be used by applications for upstream service lookups and nodes registered with Consul. Applications do not need to be modified to retrieve the service's connection information; i.e., a fully qualified domain name (FQDN) that aligns with Consul's lookup format to retrieve the relevant information.

Because you are turning on service mesh capabilities, you have the option of using service upstream or DNS. If an app is registered with Consul, it will use its local agent to query the Consul

catalog to resolve an upstream application. VM-based clients query `localhost` for services in the `.consul` domain or an `alt_domain` configured by the Consul admins.

5.3.2 DNS recursion

While clients operating locally will use Consul DNS for resolution, there will be cases where clients need to talk to off-mesh services. For this case, recursors will need to be set up to point to the upstream DNS resolvers.

5.3.3 Consul DNS on Kubernetes

Most Kubernetes distributions now use CoreDNS as a general-purpose authoritative DNS server that can provide in-cluster DNS resolution. Consul can extend its DNS capabilities to Kubernetes by integrating with CoreDNS. The "Resolve Consul DNS requests in Kubernetes" guide goes into more detail on how to configure CoreDNS in various Kubernetes distributions including OpenShift.

Organizations can use the built in `.consul` domain or choose to create a sub-domain for services catered by the Consul DNS. This sub-domain or alt-domain can be configured as a Helm value, as shown below:

```
server:
  enabled: true
  logLevel: trace
  replicas: 3
  extraConfig: |
    {
      "alt_domain": "consul.useast18.myorg.com"
    }
```

5.4 Service catalog sync

Kubernetes provides basic service discovery within a single cluster. When Consul is used solely for service discovery, there is an option to synchronize Kubernetes's service catalog with Consul's service catalog, or vice versa. Consul catalog service sync can be turned on in the Helm chart, as explained in "Service Sync for Consul on Kubernetes".

Service Mesh and Service Sync are not supported together. This is because services are automatically synced with Kubernetes when the service mesh is enabled. Additional service syncing does not need to be enabled in the Helm chart.

5.5 High-level design

As illustrated in the architecture below, a Consul deployment is designed to operate across several clouds (in this case AWS and Azure) and diverse runtime environments, including Kubernetes, Nomad, and traditional VMs. This multi-cloud, multi-runtime architecture ensures that service discovery, service mesh, and network automation capabilities are consistently delivered across diverse infrastructure environments.

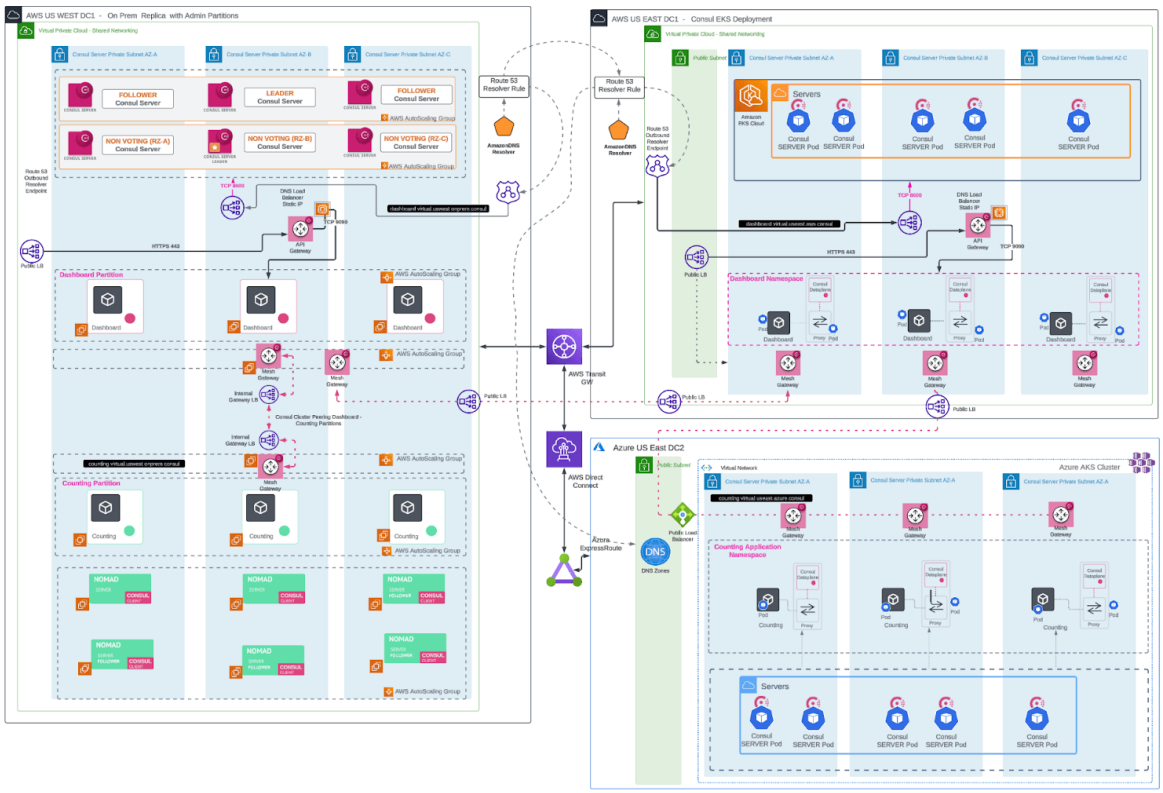


Figure 14: High level design

5.6 References

- "Consul 1.13 Introduces Cluster Peering"
- "Consul: Installation Guide - Detailed Design"
- Consul User Guide — DNS discovery
- "DNS forwarding"
- "Recursors"

6 Multi-platform application deployment

6.1 Application deployment

Once the Kubernetes clusters (EKS, AKS) are provisioned, the following YAML files define the necessary resources for deploying two applications: Counting and Dashboard. These files include configurations for ServiceAccounts, ClusterIPServers, Consul service defaults, and deployments.

6.1.1 Prerequisites

- It is recommended to create separate namespaces for each application.
- Kubernetes clusters are provisioned as per the previous steps.

6.1.2 Deploy the Counting application on AKS

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: counting
  namespace: counting
automountServiceAccountToken: true
imagePullSecrets:
  - name: registry-nonprod

---
apiVersion: v1
kind: Service
metadata:
  name: counting
  namespace: counting
  labels:
    app: counting
spec:
  type: ClusterIP
  selector:
    app: counting
  ports:
    - port: 9001
      targetPort: 9001
      protocol: TCP

---
apiVersion: consul.hashicorp.com/v1alpha1
kind: ServiceDefaults
metadata:
  name: counting
  namespace: counting
spec:
  protocol: "http"

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: counting
  namespace: counting
spec:
  replicas: 1
  selector:
    matchLabels:
      service: counting
      app: counting
  template:
    metadata:
      labels:
        service: counting
        app: counting
      annotations:
        consul.hashicorp.com/connect-inject: "true"
        k8s.v1.cni.cncf.io/networks: '[{ "name": "consul-cni" }]'
```

6.1.3 Deploy the Dashboard application on EKS

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: dashboard
  namespace: dashboard

---
apiVersion: v1
kind: Service
metadata:
  name: dashboard
  namespace: dashboard
  labels:
    app: dashboard
spec:
  type: ClusterIP
  selector:
    app: dashboard
  ports:
    - port: 8080
      targetPort: 9002
      protocol: TCP

---
apiVersion: consul.hashicorp.com/v1alpha1
kind: ServiceDefaults
metadata:
  name: dashboard
  namespace: dashboard
spec:
  protocol: "http"

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dashboard
  namespace: dashboard
  labels:
    app: dashboard
    service: dashboard
spec:
  replicas: 1
  selector:
    matchLabels:
      app: dashboard
      service: dashboard
  template:
    metadata:
      labels:
        service: dashboard
        app: dashboard
      annotations:
        consul.hashicorp.com/connect-inject: "true"
        k8s.v1.cni.cncf.io/networks: '[{ "name": "consul-cni" }]'
```

6.1.4 Dashboard service registration in AWS EC2

The example file below defines the 'dashboard' service for registration with Consul on an EC2 instance.

`dashboard.hcl`

```
service {
  name = "dashboard"
  port = 9002

  connect {
    sidecar_service {
      proxy {
        upstreams {
          destination_name = "counting"
          local_bind_port  = 9001
        }
      }
    }
  }

  check {
    name     = "HTTP Health Check"
    http     = "http://localhost:9002/health"
    interval = "10s"
    timeout  = "2s"
  }
}
```

Then, update the service in the Consul catalog:

```
consul services register dashboard.hcl
```

6.2 Nomad

This Nomad job file deploys the `counting` service in the `counting` partition, exposing port 9001 and integrating with Consul for service mesh and discovery.

> Note
>
> Admin partitions are a Consul Enterprise feature.

```
job "counting" {
  datacenters = ["nomad-dc"]
  type        = "service"
  partition   = "counting"

  group "counting-group" {
    count = 1

    network {
      port "counting" {
```

7 Deployment automation

As organizations expand, their applications and services grow in complexity and importance. These services become critical to business success necessitating continuous operation. New services will inevitably be introduced and existing ones need regular upgrades, all with minimal to zero downtime.

To achieve this level of reliability and agility, automation is essential. The industry has largely standardized around CI/CD and GitOps workflows, which HashiCorp also recommends for use with Consul.

This section covers the integration of Consul with infrastructure tools like CI/CD and ArgoCD to automate the deployment and management of services using CI/CD and GitOps workflows. You will focus on the Consul service discovery use case, which includes the deployment of applications/services, Consul client agents, service registration, and health checks.

7.1 Prerequisites

- Consul Enterprise v1.18.0 LTS.
- Access to a CI/CD tool (Jenkins, GitHub Actions, GitLab Runners, etc).
- Access to HashiCorp Cloud platform:
    - Packer
    - Terraform
    - Vault

7.2 Beginning CI/CD with Consul

Before implementing CI/CD with Consul, it is important to understand available approaches for deploying and managing software systems.

7.2.1 Managing software systems

Traditionally, software systems are managed by making in-place modifications, but this approach presents challenges. To overcome these, the industry is shifting towards immutable infrastructure, a method recommended by HashiCorp.

7.2.2 Considerations

- Teams and responsibilities

  Multiple teams are involved in the safe and reliable development and release of applications. A good understanding of the roles and responsibilities in a team will help establish ownership, along with clear channels of communication. Teams can include:

  - **Developers:** The primary consumers of CI/CD automation. Their primary responsibility is to write and maintain code for the service, which requires them to interact with VCS repositories.
  - **Infrastructure/DevOps Teams:** Responsible for setting up and configuring the CI/CD tools. This involves designing and configuring the workflows and setting up self-hosted runners. They collaborate with security teams to ensure proper security measures are taken.
  - **Security Teams:** Implement security measures for service communication and ensure that any security feature or access is aligned with the organization's security policies.
  - **Networking Teams:** Design and support the network architecture, working closely with service discovery specialists. For more details, refer to the People and process section of the Administration Guide.

- Components of a typical Consul environment

  A typical Consul environment consists of:

  - **Consul servers:** Six servers running on VMs as per the HashiCorp Recommended Architecture.
  - **Consul clients:** One per VM that is running the application or services; one per Pod when running in Kubernetes.

- **Runtime**

  Knowing the runtime environment where Consul is running will influence how CI/CD will be implemented. The runtime environment could be VMs or Kubernetes.

- **Secrets management**

  A crucial aspect is the secure retrieval and use of secrets required to execute various steps of the CI/CD workflow. Secrets to keep in mind include:

  - **Application secrets:** Those such as API keys, DB credentials, or third-party keys
  - **Consul client agent:** Consul Enterprise License key, Consul ACL tokens, Gossip Key, TLS certificates
  - HCP Packer environment variables: `HCP_CLIENT_ID`, `HCP_CLIENT_SECRET`
  - HCP Terraform API tokens: `TF_API_TOKEN`
  - **Cloud provider:** Access keys

7.3 Software management of Consul

With the above considerations in mind, group the software management of Consul into the following three categories:

1. Consul servers on VMs,
2. Services along with Consul client agents on VMs, and
3. Consul on Kubernetes.

7.3.1 Consul servers on VMs

Deploying Consul servers on VMs is HashiCorp's recommended approach. For more details, please refer to the Installation Guide.

To simplify the initial deployment, HashiCorp has created "Terraform Modules for Consul". Please contact your HashiCorp account team for access to these modules.

Changes to the Consul control plane are infrequent, so the time and effort needed to implement automation via a CI/CD pipeline may not be justifiable.

The health of the Consul servers is critical to the overall health of the cluster. For better control over how and when changes occur, it is recommended to manage it via Terraform. Moreover, Consul has features such as autopilot to help automate upgrades with Consul Enterprise easily and reliably.

7.3.2 Services along with Consul client agents on VMs

The scale of this topic is large enough to warrant its section. Continue on to the next page to learn more about this topic.

7.3.3 Consul on Kubernetes

Although HashiCorp recommends deploying the control plane Consul servers on VMs, there are scenarios where this may not be feasible. In such cases, HashiCorp fully supports deploying the Consul Control Plane and the dataplane on Kubernetes.

By leveraging the advantages of containerization, Kubernetes orchestration, and the Helm chart for Consul, HashiCorp advises using a GitOps approach to automate the deployment and management of the Consul cluster.

GitOps is a modern infrastructure and application management methodology that uses Git as the single source of truth. In a GitOps workflow, the desired state of systems—including infrastructure, applications, and configurations—is stored in a Git repository. Changes are managed through Git commits, and an automated process ensures that the actual system state aligns with the desired state defined in the repository.

Several third-party GitOps tools can facilitate this process, such as Argo CD, Flux, and Jenkins X, all of which offer up their own levels of automation and flexibility. For an example, refer to the "Deploy Consul on Kubernetes with Argo CD"article for a review of the ArgoCD platform.

8 Services with Consul client agents

Organizations may run hundreds of services across thousands of instances. Managing such a dynamic environment requires automation. HashiCorp recommends an immutable infrastructure

approach integrated with CI/CD tools.

8.1 Immutable infrastructure approach

This immutable infrastructure approach is based on the following structure:

1. **Use an application image:** All application dependencies are baked into this image. These dependencies can include the latest approved version of the OS (includes latest security patches); the application binaries & configuration files; the Consul agent binary & its configuration files; the application registration; health check configurations; security agents; scanners; logging; etc.
2. **Make use of a configuration template:** Uses the above image and other configurations to create a template that can be used by the compute nodes (VMs). In the case of AWS, you would have a launch template.
3. **Use a scaling group:** Utilizes the above configuration template to spin up compute nodes, as per its configuration. In AWS, you would use autoscaling groups.

As new versions of the application are available, the configuration templates can be updated to use the latest images via CI pipelines. Additionally, new versions of applications can be deployed by refreshing the scale groups via a CD pipeline.

8.2 Image management

HashiCorp recommends HCP Packer to automate the creation and management of images. Using the OS golden image as the source, the application image is created using Packer. This image includes:

- IT/Security-approved OS (from the golden image),
- Application binaries and configuration files,
- Consul agent binaries and configuration files,
- Application service registration, and
- Health-check configurations.

8.3 CI/CD implementation

To demonstrate a CI/CD implementation, you will deploy an example service (app-1). The following tools will be used:

Service	Actor
Cloud provider	AWS
Git repository	GitHub
Image builder	Packer
Image artifact registry	HCP Packer
Image registry	AWS AMI
Infrastructure code	HCP Terraform
CI/CD	GitHub Actions, Vault GitHub Actions
GitHub runner	Self-hosted on AWS EC2
Secrets manager	HashiCorp Vault

The diagram presents a high-level overview of the CI/CD workflow:

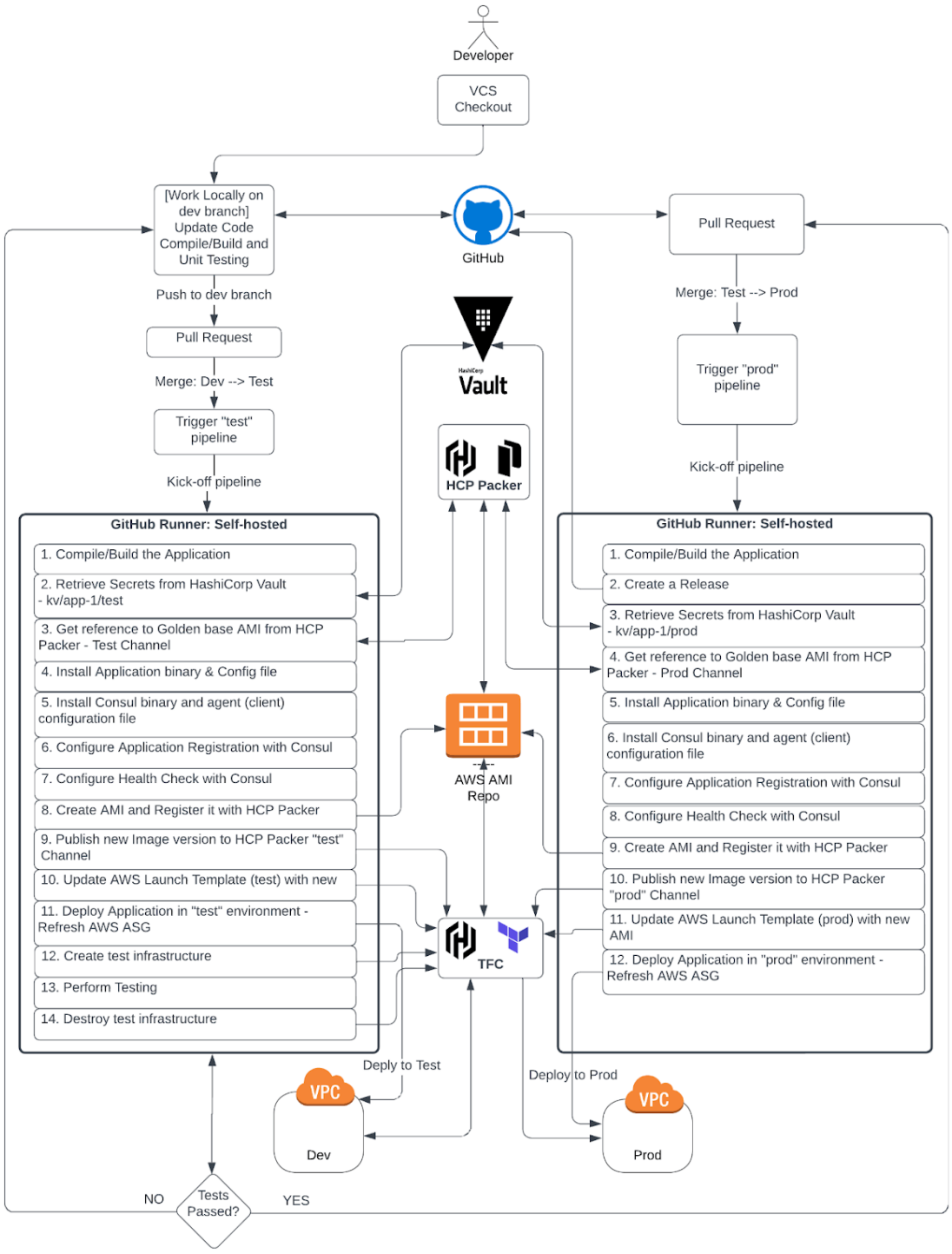


Figure 15: CI/CD workflow

Note

GitHub Actions execute workflows in runners. Runners could be GitHub-hosted or self-hosted. HashiCorp recommends using self-hosted runners to avoid external systems handling secrets, which could be a security risk. Refer to the "About self-hosted runners" document for details.

8.4 CI/CD workflow

The next set of steps and code snippets illustrate how to implement the workflow illustrated in the diagram above.

8.4.1 Checkout a "dev" branch

Developers should utilize a version-controlled repository like those found on GitHub to store application code. Code responsible for introducing new features, enhancements, bug fixes, etc... should always start on a "dev" branch. Use a local IDE to compile and test code, then push to the remote "dev" branch once the code is complete.

The structure of your code directory should be a series of folders containing appropriately separated files:

- `.github/workflows` contains code pertinent to the CI/CD workflows.
- `artifacts/` contains the compiled application binaries.
- `packer/` contains the Packer configuration that creates the images and publishes to the HCP Packer artifact registry.
- `src/` holds the application source code.
- `terraform/` is responsible for the terraform configuration files that create/update the template, along with any infrastructure for the application test environment.

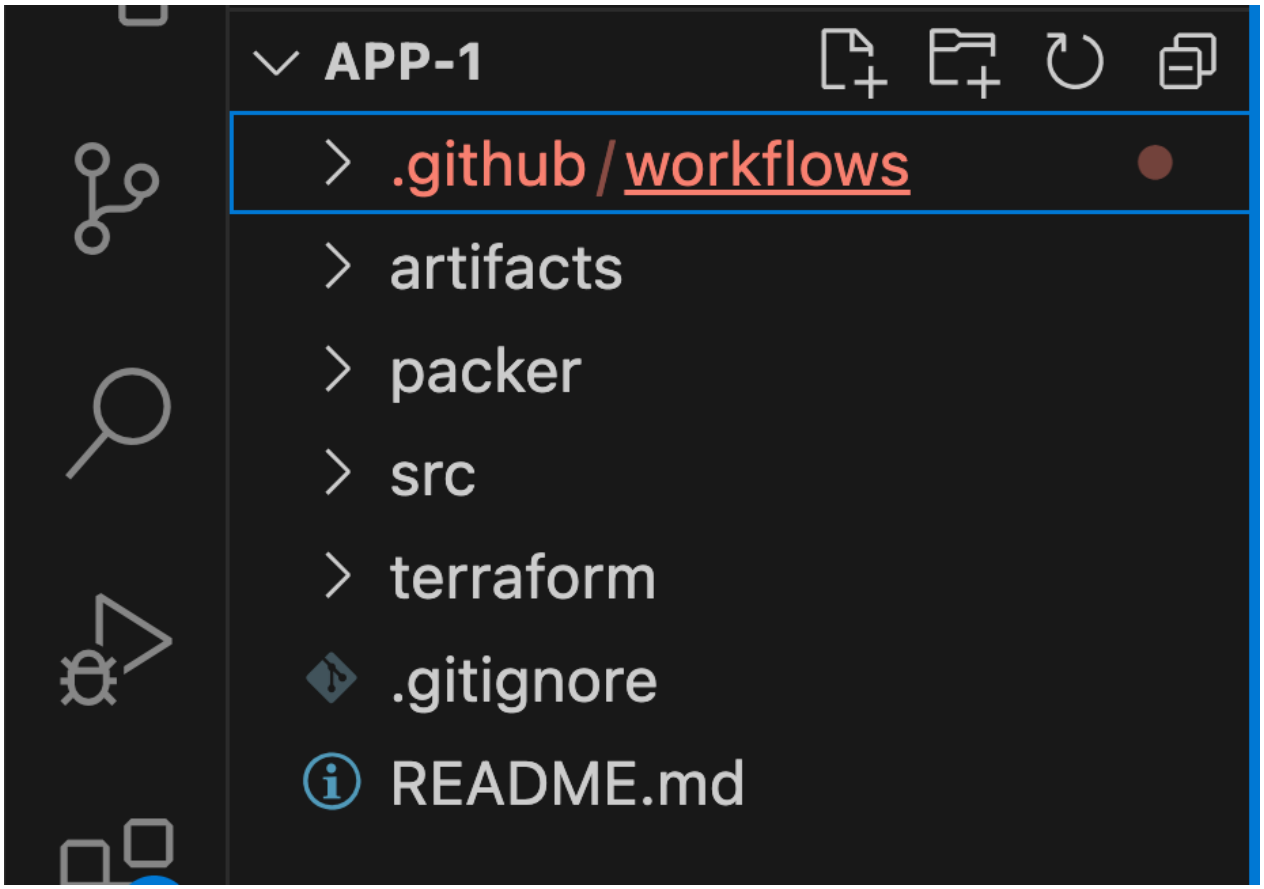


Figure 16: Directory overview

8.4.2 Pull request process

The developer responsible for the new feature creates a pull request for code review. When the code is approved, merge the "dev" branch into a "test" branch. When the code is merged, the CI/CD workflow should be kicked off automatically.

8.4.3 CI pipeline kickoff

1. **Compile/build the application**

   Build the application from the "test" branch. Save the application binaries and configuration file as a zip folder (`app-1.zip`) in the `artifacts/` directory.

`.github/workflows/app-1.yaml`

```
name: app-1 workflow
permissions:
    contents: write
    id-token: write

on:
    pull_request:
        types: [ closed ]
        branches: [ test, main ]

...
...
compile_and_build_app:
    if: github.event.pull_request.merged == true
    runs-on: self-hosted
    steps:
    - name: Checkout
        uses: actions/checkout@v4
        with:
            ref: 'refs/heads/${{ env.branch }}'
    - name: build
        run: |
            cd src
            make ${{ vars.APP_NAME }} ${{ env.branch }}
...
...
```

2. **Retrieve secrets from HashiCorp Vault**

   In this example, all secrets in Vault's KV secrets engine path `kv/data/app-1/test` are automatically saved as environment variables for the GitHub Actions. Secrets include `HCP_CLIENT_ID`, `HCP_CLIENT_SECRET`, and `TF_API_TOKEN`.

   > **Note**
   >
   > Create a Vault token with permissions to access the static secrets in Vault, and save it as a GitHub Action Secret.
   > Save the "Vault URL" and "Secrets Engine Path" values as GitHub Action variables.

   Refer to the "Vault as secrets management for Consul" tutorial for steps on creating and storing a Vault token.

   `.github/workflows/app-1.yaml`

```
...
...
- name: Retrieve Secrets from HashiCorp Vault
    id: vault
    uses: hashicorp/vault-action@v2
    with:
        url: ${{ env.vault_url }}
        tlsSkipVerify: true
        token: ${{ secrets.VAULT_TOKEN }}
        secrets: |
        ${{ env.vault_secrets_path }}/data/app-1/${{ env.branch }} * ;
...
...
```

3. **Get the golden OS image from HCP Packer**

   Extract the AMI ID from the golden OS image and use it as the source image to create a new application image.

   > **Note**
   >
   > Define the variables in a separate file variables.pkr.hcl, and set the values in a vault.auto.pkrvars.hcl file.

   `packer/app-1.pkr.hcl`

```
...
...
// Get image metadata from HCP Packer to use as the source image to create the
    new AMI
data "hcp-packer-artifact" "source-image" {
    bucket_name  = var.packer_source_image_bucket
    platform     = var.packer_source_image_platform
    region       = var.region
    version_fingerprint = "${data.hcp-packer-version.consul-client-source.
        fingerprint}"
}

data "hcp-packer-version" "consul-client-source" {
    bucket_name  = var.packer_source_image_bucket
    channel_name = var.env
}

source "amazon-ebs" "amazon-linux" {
    ami_name      = "${var.env}-${var.service_name}-${var.service_version}"
    instance_type = var.instance_type
    region        = var.region
    source_ami      = data.hcp-packer-artifact.source-image.external_identifier
    ssh_username  = var.ssh_username
}
...
...
```

4. **Install the application binary and config file**

   Run the Packer command with the branch name "test" passed in as the `env.branch` variable:

   `.github/workflows/app-1.yaml`

```
...
...
    - name: packer
        run: |
            cd packer
            packer init .
            packer build -var "env=${{ env.branch }}" .
...
...
```

   The application zip file is copied from the `artifacts/` directory to the application image and extracted into the appropriate directory.

> **Note**
>
> This depends on how your application is build and the necessary runtime dependencies, so make the required changes.

`packer/app-1.pkr.hcl`

```
...
...
build {
...
...
    // Upload Application binary
    sources = ["source.amazon-ebs.amazon-linux"]
    provisioner "file" {
        source      = "../artifacts/${var.env}/latest/${var.service_name}.zip"
        destination = "~/${var.service_name}.zip"
    }

    // Install application
    provisioner "shell" {
        inline = [
        "sudo unzip ${var.service_name}.zip",
        "sudo chmod a+x ${var.service_name}",
        "sudo cp ${var.service_name} /usr/bin/.",
        "sudo cp ${var.service_name}.service /etc/systemd/system/.",
        "sudo systemctl enable ${var.service_name}"
        ]
    }
...
...
```

5. **Install the Consul binary and agent configuration file**

   Install Consul agent via the `install-consul.sh` script, and place the Consul config template file in the `/etc/consul.d/consul.hcl.tpl` directory.

   > **Note**
   >
   > You will update the config files with actual values while updating the launch template later in the workflow.

   `packer/app-1.pkr.hcl`

```
...
...
// Install Consul Client
provisioner "shell" {
    environment_vars = [
        "CONSUL_VERSION=${var.consul_version}"
    ]
    script = "./config/scripts/install-consul.sh"
}

// Upload Consul config file
provisioner "file" {
    source      = "./config/templates/consul-client.hcl.tpl"
    destination = "/tmp/consul-client.hcl.tpl"
}

// Copy Consul Client config file
provisioner "shell" {
    inline = [
        "sudo -u consul bash -c 'cp /tmp/consul-client.hcl.tpl /etc/consul.d/
           consul.hcl.tpl'"
    ]
}
...
...
```

Use the provided bach script or other configuration management tool of your choosing to install the Consul binaries.

packer/config/scripts/install-consul.sh

```
#!/bin/bash
$ sudo yum -y update
$ sudo yum install -y yum-utils shadow-utils
$ sudo yum-config-manager --add-repo https://rpm.releases.hashicorp.com/
    AmazonLinux/hashicorp.repo
$ sudo yum -y install consul-${CONSUL_VERSION}
```

packer/config/templates/consul-client.hcl.tmpl

```
# datacenter
datacenter = "${CONSUL_DC}"

# data_dir
data_dir = "/opt/consul"

# server
server=false

# Bind addr
bind_addr = "{{ GetPrivateInterfaces | include \"network\" \"10.0.0.0/8\" | attr
    \"address\" }}"

encrypt = "${GOSSIP_KEY}"

# ACL
acl = {
    enabled = true
    default_policy = "deny"
    enable_token_persistence = true
    tokens = {
        agent = "${AGENT_TOKEN}"
    }
}

# Configure TLS
verify_incoming = true
verify_outgoing = true
verify_server_hostname = true
ca_file = "consul-agent-ca.pem"
auto_encrypt = {
    tls = true
}

# retry_join (Cloud Auto-Join)
retry_join = ["provider=aws tag_key=${CONSUL_AUTO_JOIN_TAG_KEY}
tag_value=${CONSUL_AUTO_JOIN_TAG_VALUE}"]
```

6. **Configure application registration with Consul**

   Use the provided templates to register the application with Consul. Make appropriate changes to the files so that they match your own environment configuration.

   `packer/app-1.pkr.hcl`

```
...
...
// Upload App Service registration file to Consul Node
provisioner "file" {
    source      = "./config/templates/service-registration.hcl.tmpl"
    destination = "/tmp/service-registration.hcl.tmpl"
}

// Copy Service Registration Config to Consul config dir
provisioner "shell" {
    inline = [
    "sudo -u consul bash -c 'cp /tmp/service-registration.hcl.tmpl /etc/consul.d
       /service-registration.hcl.tmpl'"
    ]
}
...
...
```

packer/config/templates/service-registration.hcl.tmpl

```
service {
    name  = "${SERVICE_NAME}"
    port  = ${SERVICE_PORT}
    token = ${SERVICE_TOKEN}
}
```

7. **Configure health checks with Consul**

   Copy the application health check configuration file to the Consul configuration directory.

   packer/app-1.pkr.hcl

```
...
...
// Copy App-1 Service Health Check template file to Consul node
provisioner "file" {
    source      = "./config/templates/health-check.hcl.tmpl"
    destination = "/tmp/health-check.hcl.tmpl"
}

// Register Service Health Check
provisioner "shell" {
    inline = [
        "sudo -u consul bash -c 'cp /tmp/health-check.hcl.tpl /etc/consul.d/
           health-check.hcl.tpl'"
    ]
}
...
...
```

This is the health check configuration template file. This could be different depending on your environment, so make the required changes.

`packer/config/templates/health-check.hcl.tmpl`

```
check = {
    id = "${SERVICE_NAME}-api"
    name = "${SERVICE_NAME}/health"
    http = "http://localhost:${SERVICE_PORT}/health"
    tls_skip_verify = true
    disable_redirects = true
    interval = "10s"
    timeout = "1s"
    token = "${SERVICE_TOKEN}"
}
```

8. Create an AMI and register it with HCP Packer

   Set the HCP Packer registry configuration with options such as `bucket_name`, `bucket_labels`, and `build_labels`.

   `packer/app-1.pkr.hcl`

```
build {
// Set HCP Packer
    hcp_packer_registry {
        bucket_name = "${var.service_name}"
        description = "${var.service_name} Image with Consul Client."

        bucket_labels = {
            "owner"               = "${var.service_name}-DevTeam"
            "os"                  = "AmazonLinux",
            "AmazonLinux-version" = "AL2",
            "Hashi-Product"       = "Consul Client"
            "Application"         = "${var.service_name}"
        }

        build_labels = {
            "build-time"   = timestamp()
            "build-source" = basename(path.cwd)
        }
    }
    ...
    ...
```

9. **Assign a new image to the HCP Packer "test" channel**

   Install Terraform on a self-hosted runner. Execute Terraform, which will assign the image to the "test" channel in HCP Packer.

   > **Note**
   >
   > Set the following secrets in an HCP Terraform variable set:
   > AWS_ACCESS_KEY_ID
   > AWS_SECRET_ACCESS_KEY
   > HCP_CLIENT_ID
   > HCP_CLIENT_SECRET

   `.github/workflows/app-1.yaml`

```
...
...
    # Install the latest version of Terraform CLI and
    # Configure the Terraform CLI configuration file with a HCP Terraform user
       API token
    - name: Setup-Terraform
        uses: hashicorp/setup-terraform@v3
        with:
            cli_config_credentials_token: ${{ env.TF_API_TOKEN }}

    - name: assign-image-to-channel
        run: |
            cd terraform/assign-image-to-channel
            terraform init
            terraform apply -var="service-name=${{ vars.APP_NAME }}" -var="env=$
                {{ env.branch }}" -auto-approve
...
...
```

10. Update the “test” AWS launch template with the new AMI

    Execute Terraform that updates with the new AMI retrieved from the HCP Packer “test” channel of the “app-1” bucket.

    `.github/workflows/app-1.yaml`

```
...
...
    needs: [ assign-image-to-packer-channel ]
    name: deploy
    runs-on: self-hosted
    steps:
    - name: terraform update aws launch template
        run: |
            cd terraform/launch-template/${{ env.branch }}
            env
            echo "Running terraform.... creating launch template..."
            terraform init
            terraform apply -auto-approve
...
...
```

    Your Terraform configuration for the launch template should look akin to what is shown in the code below:

    `terraform/launch-template/test/lt-app-1.tf`

```
...
...
data "hcp_packer_artifact" "app-1-image" {
    bucket_name = "app-1"
    platform = "aws"
    region = "us-east-1"
    channel_name = "test"
}

resource "aws_launch_template" "lt" {
    name = "lt_app-1-test"
    image_id = data.hcp_packer_artifact.app-1-image.external_identifier
    instance_type = "${var.instance_type}"
    key_name = "${var.key_pair_name}"
    iam_instance_profile {
        name = "consul-auto-join"
    }
    network_interfaces {
        security_groups = var.security_groups
    }
    user_data = filebase64("./config/app-1.sh")
}
...
...
```

Note

image_id is retrieved from the HCP Packer registry.
Inject values in the configurations/templates via user_data.
Consul ACL tokens can either be pre-generated and stored in Vault as secrets, or better yet generated dynamically using Vault's Consul secrets engine.
If an identity provided is available, Consul's auto_config is the recommended approach to distributing ACL tokens, Gossip keys, TLS certificates, and other configuration options to Consul agents.

11. **Deploy the application to the "test" environment - refresh AWS ASG**

    This executes an AWS CLI command to refresh the autoscaling group. For more details, refer to "Introducing Instance Refresh for EC2 Auto Scaling" and "Start an instance refresh using the AWS Management Console or AWS CLI".

    `.github/workflows/app-1.yaml`

```
...
...
    - name: Deploy new version - refresh ASG
        run: |
            cd terraform/asg/${{ env.branch }}
            aws autoscaling start-instance-refresh --cli-input-json file://
                config.json
...
...
```

You can customize various preferences that affect the instance refresh. For more information, refer to "Understand the default values for an instance refresh".

`terraform/asg/test/config.json`

```
{
    "AutoScalingGroupName": "asg-app-1-test",
    "Preferences": {
        "InstanceWarmup": 60,
        "MinHealthyPercentage": 50,
        "AutoRollback": false,
        "ScaleInProtectedInstances": "Ignore",
        "StandbyInstances": "Terminate"
    }
}
```

12. **Create test infrastructure**

    Create appropriate Terraform configuration file(s) to spin up a test environment as needed by your application. In this example, it is assumed that the Terraform configuration files are in the `terraform/qa/infra` directory.

    `.github/workflows/app-1.yaml`

```
...
...
    if: ${{ github.ref_name == 'test' }} # Run test in test only
    name: app-testing
    runs-on: self-hosted

    steps:
...
...
    - name: Create Test environment
        run: |
            cd terraform/qa/infra
            terraform init
            terraform apply -auto-approve
...
...
```

13. Perform testing

    Perform the necessary testing of your application. In this example, it is assumed that the testing scripts are in the `terraform/qa/testing` directory.

    `.github/workflows/app-1.yaml`

```
...
...
    - name: Testing
        if: ${{ github.ref_name == 'test' }}
        run: |
            cd terraform/qa/testing
            ./run-tests.sh
...
...
```

14. Destroy the test infrastructure

    After all tests are complete and passing, destroy the test infrastructure.

    `.github/workflows/app-1.yaml`

```
...
...
    - name: Destroy Test env
        if: ${{ github.ref_name == 'test' }}
        run: |
            cd terraform/qa/infra
            terraform destroy
...
...
```

8.4.4 Check test results

If the tests failed, a developer should go back and work on addressing any issues that arose, until all tests are in a passing state. If all of the tests pass, then the CI/CD workflow will continue.

8.4.5 Pull request procedure

Create a new pull request, this time to review and merge code from the "test" branch into a "main" branch.

After peer review is complete, approve the new changes. Then, merge the branch into "main", which will automatically kick off the workflow you walked through earlier, this time utilizing the "main" branch.

8.4.6 Kick off the CI pipeline

The CI/CD workflow steps are similar to what is discussed above, but this time:

- The "main" branch is being used,
- An additional "Create a release" step is being executed, and
- Test-related steps are not required, and thus not executed.

A sequence of steps will take place:

1. Compile/Build the application.
2. Create a release.

   The 'Create a release' step triggers a release. This creates a tag and uploads the application artifacts to GitHub as a release.

`.github/workflows/app-1.yaml`

```
...
...
release_app:
    needs: [ compile_and_build_app ]
    if: ${{ github.ref_name == 'main' }} # Execute only when merging to main
        branch
    runs-on: self-hosted
    steps:
    - name: Create a Release & Upload artifacts
        id: create_release
        uses: ncipollo/release-action@v1
        with:
            tag: v-${{ github.run_number }}
            name: Release v-${{ github.run_number }}
            artifacts: "./artifacts/${{ env.branch }}/latest/${{ vars.APP_NAME
                }}.zip"
            makeLatest: "true"
...
...
```

3. Retrieve secrets from HashiCorp Vault (`kv/app-1/prod`).
4. Get a reference to the golden OS image from HCP Packer - 'prod' channel.
5. Install the application binary and configuration file.
6. Install the Consul binary and agent configuration file.
7. Configure application registration with Consul.
8. Configure health checks with Consul.
9. Create an AMI and register it with HCP Packer.
10. Assign the new image to HCP Packer - 'prod' channel.
11. Update the "prod" AWS launch template with the new AMI.
12. Deploy your application in the "prod" environment - refresh AWS ASG.

Upon successful completion of the pipeline, the application in production will be running the latest version of the application, deployed on the most recently approved version of the operating system. Any future updates to the application will automatically trigger this CI/CD pipeline, ensuring seamless updates to the production environment.

8.5 References

- "Build a golden image pipeline with HCP Packer"
- "Terraform: Integration Guide for HCP Packer"

9 Changelog

This page records notable changes to the Consul User Guide.

9.1 2026-08

- Restructured legacy Consul HVD content into the HVD 2.0 Consul User Guide as part of the Installation / Administration / User guide migration.

9.2 2026-01

- Update meta descriptions
- Updated credits and contributors

9.3 2025-06

- Updated credits and contributors
- Use consistent guide names and descriptions
- Standardized operating guide naming conventions

9.4 2024-11

- Beta Release

10 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

List of key team members who contributed to the development of this guide. Hailing from various functional areas and representing diverse departments within HashiCorp, these experienced professionals have played pivotal roles in writing, editing, reviewing, and ensuring the completion of this material. Their collective efforts have been instrumental in shaping the final output.

- Abhishek Tiwari
- Adam Cavaliere
- Austin Workman
- Blake Covarrubias
- Çetin Ardal
- Chris Dunlap
- Cooper Melgreen
- David Cañadillas
- David Canadillas
- Gerald Dykeman
- Hari Sankaran
- Jan Repnak
- Josh Wolfer
- Kalen Arndt
- Kulpreet Singh
- Luke Kysow
- Mara Hammond
- Mark Lewis
- Pablo Diaz
- Prabhjit Singh
- Sean Doyle
- Srini Nagaraju
- Tu Nguyen